

**TRƯỜNG ĐẠI HỌC ĐÀ LẠT
KHOA CÔNG NGHỆ THÔNG TIN**



**BÁO CÁO TỔNG KẾT HỌC PHẦN
HỌC PHẦN: CÁC CÔNG NGHỆ MỚI
TRONG PHÁT TRIỂN PHẦN MỀM
ĐỀ TÀI: PHÁT TRIỂN NỀN TẢNG DU LỊCH THÔNG MINH
DỰA THEO THỜI TIẾT VÀ TÍCH HỢP TRỢ LÝ ẢO**

Giảng viên hướng dẫn: KS. Nguyễn Trọng Hiếu

Sinh viên thực hiện: 2212375 - Triệu Quang Học

Lâm Đồng, tháng 05 năm 2026

[illegible]

LỜI CẢM ƠN

Trong quá trình hoàn thành dự án này, em xin bày tỏ lòng biết ơn sâu sắc đến thầy Nguyễn Trọng Hiếu – giảng viên hướng dẫn của em trong học phần này. Thầy đã tận tình hướng dẫn, cung cấp kiến thức và hỗ trợ em xuyên suốt quá trình thực hiện dự án. Thầy không chỉ là người truyền đạt kiến thức chuyên môn, mà còn là người đã tận tình đồng hành, chỉ bảo và định hướng cho nhóm em ngay từ những bước đi đầu tiên trong quá trình xác định đề tài cho đến khi hoàn thiện sản phẩm cuối cùng. Những buổi hướng dẫn, những góp ý sâu sắc, mang tính xây dựng cao và những nhận xét quý báu của thầy về cả khía cạnh lý thuyết lẫn thực tiễn đã đóng vai trò then chốt trong việc giúp em vượt qua mọi khó khăn, chuẩn hóa phương pháp nghiên cứu, và nâng cao đáng kể chất lượng của sản phẩm này.

Em xin bày tỏ lòng biết ơn chân thành đến Ban Lãnh đạo Khoa, Ban Chủ nhiệm Bộ môn cùng toàn thể quý thầy cô tại trường đã tạo điều kiện thuận lợi về cơ sở vật chất, tài liệu học tập và môi trường nghiên cứu tốt nhất để em có thể tập trung thực hiện dự án. Em vô cùng cảm kích sự hỗ trợ và động viên kịp thời từ quý thầy cô trong suốt thời gian qua.

Em cũng xin gửi lời cảm ơn đặc biệt đến các bạn sinh viên, đặc biệt là các bạn trong lớp và những người bạn đã cùng nhau học tập, nghiên cứu. Sự nhiệt tình hỗ trợ, những cuộc thảo luận thẳng thắn, những ý kiến phản biện đa chiều và sự chia sẻ kinh nghiệm quý báu từ các bạn đã giúp em nhìn nhận vấn đề một cách toàn diện hơn, từ đó hoàn thiện hơn nữa những thiếu sót trong dự án của mình.

Cuối cùng, em xin chân thành cảm ơn tất cả những cá nhân, tổ chức đã trực tiếp hoặc gián tiếp hỗ trợ trong suốt quá trình thực hiện dự án này, từ việc cung cấp dữ liệu, chia sẻ thông tin chuyên ngành, đến sự động viên tinh thần. Sự giúp đỡ, lòng tin và động lực từ mọi người chính là nguồn sức mạnh to lớn, thúc đẩy em cố gắng, vượt qua mọi thử thách để hoàn thành xuất sắc đồ án chuyên ngành.

Ưm xin kính chúc thầy cô, quý vị và các bạn luôn dồi dào sức khỏe, hạnh phúc và thành công trên con đường sự nghiệp và học vấn.

Trân trọng!

MỤC LỤC

DANH MỤC HÌNH ẢNH.....	1
DANH MỤC BẢNG	2
CHƯƠNG 1: TỔNG QUAN	1
1.1. Bối cảnh và Lý do chọn đề tài.....	1
1.2. Giới thiệu dự án Dalat SmartRoute AI	2
1.3. Mục tiêu của dự án.....	2
1.3.1. Mục tiêu chính về mặt nghiệp vụ	2
1.3.2. Mục tiêu về mặt kỹ thuật	3
1.3.3. Mục tiêu về mặt sản phẩm ứng dụng.....	3
1.4. Phạm vi của dự án	4
1.4.1. Các thành phần hệ thống nằm trong phạm vi phát triển.....	4
1.4.2. Các thành phần ngoài phạm vi dự án (Giới hạn loại trừ)	4
CHƯƠNG 2: CÔNG NGHỆ VÀ CÔNG CỤ PHÁT TRIỂN	5
2.1. Nền tảng Frontend và Kiến trúc cốt lõi.....	5
2.1.1. Framework Next.js và Kiến trúc App Router tiên tiến.....	5
2.1.2. Hệ thống kiểu tĩnh với TypeScript	6
2.1.3. Thiết kế giao diện hiện đại với Tailwind CSS.....	6
2.2. Nền tảng Backend và Quản trị Cơ sở dữ liệu	7
2.2.1. Nền tảng Backend-as-a-Service (BaaS) - Supabase	7
2.2.2. Quản lý xác thực (Auth) và Bảo mật cấp độ hàng (RLS).....	7
2.2.3. Tương tác cơ sở dữ liệu bằng Prisma ORM	8
2.3. Tích hợp Trí tuệ nhân tạo và Dữ liệu ngoại vi.....	9
2.3.1. Dịch vụ thời tiết OpenWeather API	9
2.3.2. Nền tảng Trí tuệ nhân tạo Generative AI (Google Gemini).....	9
2.4. Công cụ Đóng gói và Triển khai (DevOps).....	10
2.4.1. Ảo hóa ứng dụng với công nghệ Docker.....	10
2.4.2. Triển khai máy chủ (VPS) và Chứng chỉ bảo mật (SSL/TLS).....	10
CHƯƠNG 3: KIẾN TRÚC HỆ THỐNG.....	12
3.1. Phân tích và đặc tả yêu cầu hệ thống	12
3.1.1. Đặc tả yêu cầu chức năng (Functional Requirements - FR).....	12
3.1.2. Đặc tả yêu cầu phi chức năng (Non-functional Requirements - NFR)	14

3.2. Phân tích Tác nhân và Use Case	15
3.2.1. Xác định Tác nhân (Actors).....	15
3.2.2. Biểu đồ Use Case tổng quát.....	16
3.2.3. Đặc tả Kịch bản Use Case lỗi (Use Case Specifications).....	17
3.3. Thiết kế Kiến trúc tổng thể hệ thống.....	20
3.3.1. Lớp Trình bày (Presentation Layer/Client Interface)	21
3.3.2. Lớp Logic Ứng dụng và Dịch vụ (Application & Service Layer).....	21
3.3.3. Lớp Truy cập Dữ liệu (Data Access Layer - DAL).....	21
3.4. Mô hình hóa và Thiết kế Cơ sở dữ liệu (ERD).....	22
3.4.1. Thiết kế bảng dữ liệu (Data Dictionary).....	23
3.4.2. Thiết lập ràng buộc Toàn vẹn tham chiếu (Referential Integrity)	27
3.5. Thiết kế Hệ thống Giao diện lập trình ứng dụng (API Specifications)	27
3.5.1. Endpoint Xác thực và Cấp phát Token (Login).....	27
3.5.2. Endpoint Khai thác Địa điểm theo Thời tiết (Weather Recommendations).28	
3.5.3. Endpoint Trò chuyện với Trợ lý AI (Generative AI Chat).....	28
3.6. Thiết kế các luồng hoạt động nghiệp vụ chi tiết (Sequence Flows)	29
3.6.1. Luồng xử lý phân tích ngữ cảnh của Trợ lý AI (Google Gemini Flow)	29
3.6.2. Luồng giao dịch tạo lập và cập nhật Đánh giá cộng đồng (Review System)	30
3.7. Thiết kế Giao diện người dùng (UI/UX).....	32
CHƯƠNG 4: XÂY DỰNG VÀ TRIỂN KHAI HỆ THỐNG	33
4.1. Tổ chức cấu trúc mã nguồn (Source Code Organization).....	33
4.1.1. Phân hệ Giao diện và Định tuyến (Frontend & Routing)	33
4.1.2. Phân hệ Dịch vụ Backend (API Layer)	33
4.1.3. Phân hệ Tương tác Dữ liệu (Database ORM).....	33
4.2. Triển khai Cơ sở dữ liệu trên môi trường Supabase	34
4.2.1. Cấu hình Chuỗi kết nối (Connection Pooling)	34
4.2.2. Triển khai Lược đồ và Nạp dữ liệu mẫu (Migration & Seeding).....	34
4.3. Đóng gói ứng dụng với Docker (Containerization)	35
4.3.1. Tối ưu hóa với Dockerfile (Multi-stage Build)	35
4.3.2. Quản lý luồng với Docker Compose	36
4.4. Triển khai Hệ thống và Tích hợp Liên tục (CI/CD Production Deployment)	36

4.4.1. Quy trình CI/CD và Serverless Deployment	36
4.4.2. Quản lý Biến môi trường và Chứng chỉ Bảo mật (SSL/TLS)	36
4.5. Ứng dụng Trí tuệ nhân tạo (AI Tools) trong quy trình phát triển.....	37
4.6. Kết quả triển khai giao diện người dùng (UI/UX Results)	39
4.6.1. Trang chủ và Khởi tìm kiếm trung tâm (Hero Section & Search).....	40
4.6.2. Giao diện khám phá và Gợi ý theo thời tiết (Weather-based Recommendations)	40
4.6.3. Giao diện Chi tiết địa điểm và Tương tác cộng đồng (Place Details & Reviews)	41
4.6.4. Giao diện Trợ lý ảo thông minh (AI Chatbot Widget)	42
4.6.5. Bảng điều khiển Quản trị viên (Admin Dashboard).....	43
CHƯƠNG 5: TỔNG KẾT, ĐÁNH GIÁ VÀ HƯỚNG PHÁT TRIỂN	45
5.1. Tổng kết những kết quả đạt được.....	45
5.2. Đánh giá ưu điểm của hệ thống.....	45
5.3. Hạn chế và các vấn đề còn tồn đọng	46
5.4. Hướng phát triển trong tương lai	47
TÀI LIỆU THAM KHẢO	49

DANH MỤC HÌNH ẢNH

Bảng 3.1: Danh sách Actors trong hệ thống.....	15
Hình 3.2: Sơ đồ kiến trúc hệ thống.....	20
Hình 3.3: Lược đồ quan hệ thực thể (ERD)	23
Hình 4.1: Ảnh chụp màn hình Trang chủ (Home Page) của hệ thống	40
Hình 4.2: Ảnh chụp màn hình trang Danh sách địa điểm (Places List)	41
Hình 4.3: Ảnh chụp màn hình trang Chi tiết địa điểm (Detail Page).....	41
Hình 4.4: Ảnh chụp màn hình khu vực Bình luận và Đánh giá (Review Form).....	42
Hình 4.5: Ảnh chụp màn hình cửa sổ Chatbot AI đang tương tác trả lời câu hỏi của người dùng.....	43
Hình 4.6: Ảnh chụp màn hình Bảng điều khiển Admin quản lý danh sách địa điểm hoặc người dùng	44

DANH MỤC BẢNG

Bảng 3.1: Danh sách Actors trong hệ thống.....	15
Bảng 3.2: Thiết kế bảng Thông tin tài khoản người dùng.....	24
Bảng 3.3: Thiết kế bảng Danh mục phân loại	24
Bảng 3.4: Thiết kế bảng Điểm đến du lịch.....	26
Bảng 3.5: Thiết kế bảng Đánh giá của cộng đồng.....	26
Bảng 3.6: Thiết kế bảng Lưu trữ Yêu thích.....	27
Bảng 4.1: Minh chứng ứng dụng AI trong phát triển dự án.....	39

CHƯƠNG 1: TỔNG QUAN

1.1. Bối cảnh và Lý do chọn đề tài

Trong kỷ nguyên số hóa hiện nay, ngành du lịch đang chứng kiến những bước chuyển mình mạnh mẽ nhờ vào sự giao thoa của công nghệ thông tin và nhu cầu trải nghiệm mang tính cá nhân hóa ngày càng cao của du khách. Thành phố Đà Lạt, với vị thế là một trong những trung tâm du lịch trọng điểm của Việt Nam, luôn thu hút lượng lớn khách tham quan trong và ngoài nước nhờ khí hậu ôn đới và cảnh quan tự nhiên đặc thù. Tuy nhiên, đặc trưng khí hậu của vùng cao nguyên này thường có biến động lớn, thời tiết nắng mưa thất thường theo từng khung giờ trong ngày, gây ảnh hưởng trực tiếp và không nhỏ tới lộ trình di chuyển cũng như trải nghiệm thực tế của du khách. Việc tìm kiếm một địa điểm tham quan, ẩm thực hay không gian thư giãn phù hợp với điều kiện thời tiết thực tế tại một thời điểm cụ thể thường tiêu tốn nhiều thời gian và dễ dẫn tới những trải nghiệm không như mong đợi do thiếu hụt nguồn thông tin cập nhật theo ngữ cảnh thời gian thực.

Đồng thời, xét trên góc độ kỹ nghệ phần mềm và xu hướng phát triển ứng dụng hiện đại thuộc môn học Các công nghệ mới trong phát triển phần mềm, quy trình xây dựng các sản phẩm công nghệ đang có những bước tiến vượt bậc. Sự dịch chuyển từ các kiến trúc truyền thống sang mô hình tối ưu hiệu năng phân phối dữ liệu thông qua cơ chế kết xuất phía máy chủ (Server-Side Rendering - SSR) của framework Next.js [1] đã đặt ra những tiêu chuẩn mới về tốc độ và trải nghiệm người dùng. Bên cạnh đó, việc áp dụng các mô hình Nền tảng dịch vụ đám mây tích hợp (Backend-as-a-Service) như Supabase [2] giúp đơn giản hóa và tăng tốc quá trình quản lý định danh và cơ sở dữ liệu, kết hợp với tầng giao tiếp dữ liệu an toàn thông qua Prisma ORM [3]. Đặc biệt, sự bùng nổ của Trí tuệ nhân tạo sinh tạo (Generative AI) [4] đã mở ra cơ hội tích hợp các hệ thống trợ lý thông minh có năng lực tư vấn, phản hồi tự nhiên dựa trên tập dữ liệu miền chuyên biệt thay vì các câu lệnh phản hồi tĩnh thông thường.

Nhằm giải quyết triệt để bài toán thực tiễn về tối ưu hóa trải nghiệm du lịch tại địa phương, đồng thời nghiên cứu, làm chủ và ứng dụng đồng bộ các giải pháp công nghệ tiên tiến nhất hiện nay vào quy trình phát triển ứng dụng toàn phần (fullstack), đề tài "**Phát triển nền tảng du lịch thông minh dựa theo thời tiết và tích hợp trợ lý ảo**" đã được lựa chọn để triển khai.

1.2. Giới thiệu dự án Dalat SmartRoute AI

Dalat SmartRoute AI là một nền tảng ứng dụng web toàn phần (fullstack) hướng tới việc cung cấp dịch vụ thông minh hỗ trợ khách tham quan khám phá, định tuyến và lựa chọn điểm đến tối ưu tại thành phố Đà Lạt. Hệ thống hoạt động dựa trên việc phân tích mối tương quan động giữa dữ liệu ngữ cảnh thời tiết thực tế, hệ thống định vị không gian địa lý của các địa điểm và các luồng thông tin đóng góp, phản hồi từ cộng đồng người dùng thực tế. Điểm nhấn kiến trúc của hệ thống nằm ở việc tích hợp sâu khả năng tương tác trực tiếp với một trợ lý trí tuệ nhân tạo (AI Assistant), giúp phân tích yêu cầu tự nhiên của du khách để đưa ra các đề xuất lịch trình cá nhân hóa sâu sắc và mang giá trị tư vấn thực tiễn cao.

Về mặt cấu trúc công nghệ, hệ thống được phân rã thành các lớp kiến trúc rõ ràng bao gồm:

- Lớp trình bày (Presentation Layer): Được xây dựng dựa trên framework Next.js sử dụng kiến trúc App Router tiên tiến để tối ưu hóa SEO và hiệu năng truyền tải trang qua cơ chế kết xuất hỗn hợp.
- Lớp dịch vụ Backend (Application Layer): Đảm nhiệm xử lý toàn bộ logic nghiệp vụ điều hướng, quản lý định danh và cung cấp hệ thống endpoint RESTful an toàn.
- Lớp truy cập dữ liệu (Data Access Layer): Sử dụng công cụ Prisma ORM đóng vai trò như một bộ dụng cụ truy vấn mạnh mẽ, giúp ánh xạ dữ liệu và giảm thiểu rủi ro bảo mật.
- Lớp lưu trữ dữ liệu (Database Layer): Vận hành trên nền tảng cơ sở dữ liệu quan hệ PostgreSQL thông qua dịch vụ đám mây quản lý của Supabase.
- Hệ thống tích hợp ngoại vi (External Integrations): Liên kết trực tiếp với dịch vụ khí tượng OpenWeather API [5] để thu thập trạng thái thời tiết thời gian thực và kết nối API Google Generative AI (mô hình GEMINI) để xử lý ngôn ngữ tự nhiên và sinh nội dung gợi ý thông minh.

1.3. Mục tiêu của dự án

1.3.1. Mục tiêu chính về mặt nghiệp vụ

- Nghiên cứu và xây dựng thành công một giải pháp công nghệ số hóa du lịch có năng lực phân tích thông minh, tự động thực hiện cá nhân hóa các đề xuất điểm

đến dựa trên sự thay đổi của điều kiện thời tiết thực tế (nhiệt độ, mưa, nắng) phối hợp với các sở thích đặc thù của từng nhóm đối tượng du khách.

- Nâng cao một cách bền vững chất lượng trải nghiệm du lịch của du khách tại Đà Lạt, giúp họ chủ động ứng phó với các kịch bản thời tiết bất lợi.
- Thiết lập một hệ sinh thái dữ liệu số tương tác lấy cộng đồng làm trung tâm (community-driven approach), cho phép tích lũy tri thức du lịch bản địa thông qua các hoạt động viết đánh giá (review), chấm điểm sao trực quan và lưu trữ danh mục yêu thích cá nhân.

1.3.2. Mục tiêu về mặt kỹ thuật

- Xây dựng một kiến trúc hệ thống phần mềm full-stack chuẩn mực, đảm bảo tính đóng gói tốt, có khả năng mở rộng linh hoạt (scalable), dễ bảo trì và giảm thiểu sự phụ thuộc lẫn nhau giữa các tính năng (tính độc lập về mặt tính năng cao).
- Đảm bảo tính an toàn thông tin và toàn vẹn dữ liệu cho toàn bộ hệ thống thông qua việc triển khai cơ chế xác thực dựa trên mã hóa token JWT (JSON Web Token) kết hợp hệ thống kiểm soát truy cập và quản lý quyền hạn nghiêm ngặt theo vai trò của người dùng (Role-Based Access Control - RBAC).
- Tối ưu hóa tài nguyên phần cứng và giảm thiểu chi phí vận hành hạ tầng máy chủ bằng cách khai thác triệt để các dịch vụ đám mây được quản trị sẵn (managed services) của nền tảng Supabase.

1.3.3. Mục tiêu về mặt sản phẩm ứng dụng

- Cung cấp tính năng lọc và xếp hạng địa điểm thông minh theo thời gian thực dựa trên trạng thái khí tượng thực tế tại Đà Lạt: Ưu tiên tối đa các địa điểm trong nhà (indoorSuitable = true) khi trời có mưa và điều hướng tới các hoạt động trải nghiệm không gian thoáng đãng ngoài trời khi thời tiết nắng ráo.
- Tích hợp thành công mô hình ngôn ngữ lớn để vận hành một trợ lý ảo thông minh có khả năng truy xuất, tổng hợp và phân tích dữ liệu miền chuyên biệt (domain-specific data) được lưu trữ cục bộ phối hợp cùng các đánh giá cộng đồng để đưa ra câu trả lời tư vấn lịch trình chính xác.
- Cho phép người dùng thiết lập, quản lý và tùy biến không gian cá nhân của mình bao gồm việc theo dõi lịch sử tương tác, quản lý danh sách địa điểm yêu thích và thay đổi các tùy chỉnh về sở thích trải nghiệm.

1.4. Phạm vi của dự án

1.4.1. Các thành phần hệ thống nằm trong phạm vi phát triển

Dự án tập trung hoàn thiện các thành phần cốt lõi thuộc ba tầng kiến trúc chính:

- Giao diện người dùng hướng khách hàng (Client UI): Thiết kế và tối ưu hóa hệ thống các trang chức năng bao gồm: trang chủ duyệt danh sách địa điểm, công cụ tìm kiếm nâng cao phối hợp bộ lọc danh mục, trang hiển thị thông tin chi tiết địa điểm, phân hệ đăng ký/đăng nhập tài khoản bảo mật, và không gian quản lý hồ sơ cá nhân của người dùng.
- Hệ thống API Backend (RESTful Endpoints): Triển khai toàn bộ các tuyến đường dẫn API phục vụ xử lý luồng xác thực người dùng, thực hiện các thao tác quản trị dữ liệu (CRUD) đối với các thực thể người dùng, địa điểm, danh mục, bài viết đánh giá từ cộng đồng, danh sách yêu thích, cũng như các endpoint tiếp nhận và tiền xử lý dữ liệu từ OpenWeather API và Google Generative AI.
- Tầng quản trị và lưu trữ dữ liệu (Data & Infrastructure Layer): Thiết kế lược đồ quan hệ thực thể cơ sở dữ liệu thông qua Prisma Schema, xây dựng các bộ mã lệnh khởi tạo dữ liệu mẫu (seed script) và các tệp kiểm soát lịch sử thay đổi cấu trúc database (migration logs) vận hành trực tiếp trên PostgreSQL của Supabase.

1.4.2. Các thành phần ngoài phạm vi dự án (Giới hạn loại trừ)

Nhằm tập trung nguồn lực hoàn thiện các tính năng cốt lõi theo đúng yêu cầu thời hạn và mục tiêu kỹ thuật đặt ra, dự án chủ động loại trừ các thành phần sau:

- Không xây dựng hệ thống thanh toán điện tử trực tuyến hoặc các phân hệ quản lý giao dịch tài chính phức tạp giữa người dùng và các địa điểm.
- Dự án hiện tại chỉ tập trung nghiên cứu và tối ưu hóa hiển thị trên nền tảng ứng dụng Web (Web Application), không bao gồm việc phát triển các ứng dụng di động độc lập (Mobile Native App) trên các hệ điều hành iOS hay Android.
- Hệ thống không phát triển một giao diện trang quản trị nội dung (CMS Dashboard) độc lập và đồ sộ dành cho quản trị viên; mọi thao tác cấu hình, kiểm duyệt người dùng hay điều chỉnh thông tin hệ thống sâu ở giai đoạn này sẽ được thực hiện trực tiếp thông qua hệ thống API endpoints bảo mật và bảng quản trị cơ sở dữ liệu trực tiếp của Supabase.

CHƯƠNG 2: CÔNG NGHỆ VÀ CÔNG CỤ PHÁT TRIỂN

2.1. Nền tảng Frontend và Kiến trúc cốt lõi

2.1.1. Framework Next.js và Kiến trúc App Router tiên tiến

Next.js là một framework mã nguồn mở mạnh mẽ được xây dựng trên nền tảng thư viện React.js, do Vercel phát triển và bảo trợ. Khác với các ứng dụng Single Page Application (SPA) truyền thống của React.js thường gặp hạn chế về Tối ưu hóa công cụ tìm kiếm (SEO) và thời gian tải trang ban đầu (Initial Load Time), Next.js giải quyết triệt để vấn đề này bằng cách hỗ trợ đa dạng các phương pháp kết xuất trang: Server-Side Rendering (SSR), Static Site Generation (SSG) và Incremental Static Regeneration (ISR) [1].

Trong dự án này, hệ thống áp dụng phiên bản Next.js mới nhất với kiến trúc App Router. Đây là một bước ngoặt lớn về mặt kiến trúc so với Pages Router thế hệ cũ, nổi bật với việc tích hợp sâu mô hình React Server Components (RSC).

- Đặc tính kỹ thuật và lý do lựa chọn: Server Components cho phép kết xuất (render) các thành phần giao diện trực tiếp trên máy chủ trước khi gửi dưới dạng HTML thuần túy xuống máy khách. Điều này giúp loại bỏ một lượng lớn mã JavaScript không cần thiết khỏi gói tải về (bundle size) của trình duyệt.
- Vai trò trong dự án: Đối với nền tảng Dalat SmartRoute AI, tốc độ tải trang và SEO là hai yếu tố sống còn để thu hút du khách tiếp cận nền tảng qua các công cụ tìm kiếm. Next.js Server Components được sử dụng làm bộ khung để kết xuất các trang danh sách địa điểm, trang chi tiết điểm đến và các dữ liệu tĩnh. Ngược lại, Client Components (được khai báo qua directive "use client") chỉ được khoanh vùng ở những khu vực đòi hỏi tương tác trạng thái (stateful) cao từ người dùng như: thanh tìm kiếm (Search Bar), hệ thống lọc, cửa sổ trò chuyện với AI (Chat Widget) và các biểu mẫu (Forms). Sự phân tách rõ ràng này giúp tối ưu hóa hiệu năng tổng thể một cách vượt trội.
- Tích hợp Backend: Hơn thế nữa, Next.js cung cấp cơ chế Route Handlers (thông qua các tệp route.ts đặt trong thư mục src/app/api/). Dự án tận dụng tính năng này để xây dựng toàn bộ hệ thống API nội bộ, bao bọc (wrap) các truy vấn cơ sở dữ liệu và gọi API ngoại vi (AI, Thời tiết). Nhờ đó, dự án đạt được sự đồng nhất trong môi trường phát triển (Monorepo), loại bỏ hoàn toàn sự phức tạp của việc

phải duy trì và triển khai một máy chủ backend độc lập bằng Express.js hay NestJS.

2.1.2. Hệ thống kiểu tĩnh với TypeScript

TypeScript là một ngôn ngữ lập trình mã nguồn mở, bản chất là một siêu tập hợp (superset) của JavaScript do Microsoft phát triển. Đặc điểm nổi bật nhất của TypeScript là việc bổ sung hệ thống kiểm tra kiểu tĩnh (static type checking) mạnh mẽ vào quá trình viết mã [6].

- Vai trò trong dự án: Sự phức tạp của hệ thống quản lý du lịch đa tương tác – nơi dữ liệu luân chuyển liên tục giữa Client, Server, CSDL và các API của bên thứ ba – đặt ra bài toán lớn về tính toàn vẹn của cấu trúc dữ liệu. TypeScript được áp dụng nghiêm ngặt trên toàn bộ mã nguồn của dự án (cả Frontend và Backend).
- Nhờ việc định nghĩa các Interface và Type cho các thực thể như User, Place, Review, và cấu trúc JSON phức tạp trả về từ OpenWeather API, các lỗi tiềm ẩn (như truy cập thuộc tính không tồn tại, sai kiểu dữ liệu) được trình biên dịch (compiler) phát hiện và báo lỗi ngay lập tức trong lúc lập trình (compile-time), thay vì để hệ thống đổ vỡ ở thời gian chạy thực tế (run-time). Hơn nữa, TypeScript kết hợp với trình soạn thảo VS Code mang lại trải nghiệm tự động hoàn thiện mã (IntelliSense) tuyệt vời, tăng tốc độ phát triển và giúp mã nguồn trở thành một tài liệu tự giải thích (self-documenting) hiệu quả.

2.1.3. Thiết kế giao diện hiện đại với Tailwind CSS

Tailwind CSS là một utility-first CSS framework (bộ khung CSS ưu tiên các lớp tiện ích), cho phép nhà phát triển xây dựng các giao diện web tùy chỉnh và linh hoạt một cách nhanh chóng bằng cách tổ hợp các lớp tiện ích trực tiếp vào cấu trúc mã HTML hoặc JSX [7].

- Lý do lựa chọn: Thay vì phải định nghĩa các tệp CSS riêng biệt công kênh với các phương pháp đặt tên phức tạp (như BEM), Tailwind CSS cung cấp một hệ thống thiết kế (design system) chuẩn mực ngay từ đầu. Framework này đi kèm với một trình biên dịch thông minh, tự động loại bỏ các đoạn CSS không được sử dụng (purging) trong quá trình build production, giúp tệp CSS cuối cùng có dung lượng cực kỳ nhỏ gọn (thường dưới 10KB).
- Vai trò trong dự án: Trong bối cảnh người dùng tra cứu thông tin du lịch chủ yếu qua thiết bị di động, tính đáp ứng (Responsive Web Design) là bắt buộc. Hệ thống

sử dụng các tiền tố kích thước màn hình của Tailwind (như sm:, md:, lg:) để thiết kế giao diện linh hoạt, hiển thị mượt mà trên mọi thiết bị. Bên cạnh đó, các thành phần UI được chuẩn hóa đồng bộ về tông màu (color palette), khoảng cách (spacing), và hiệu ứng thị giác (shadows, transitions) nhằm mang lại trải nghiệm thân thiện, trực quan và giữ chân du khách lâu hơn trên hệ thống.

2.2. Nền tảng Backend và Quản trị Cơ sở dữ liệu

2.2.1. Nền tảng Backend-as-a-Service (BaaS) - Supabase

Supabase là một nền tảng Backend-as-a-Service mã nguồn mở mạnh mẽ, nổi lên như một giải pháp thay thế tối ưu cho nền tảng Firebase của Google [2]. Điểm khác biệt lớn nhất và cũng là thế mạnh cốt lõi của Supabase nằm ở việc nó được xây dựng hoàn toàn trên nền tảng hệ quản trị cơ sở dữ liệu quan hệ mã nguồn mở mạnh mẽ nhất thế giới là PostgreSQL.

- Vai trò trong dự án: Việc xây dựng từ đầu (from scratch) một hạ tầng cơ sở dữ liệu, thiết lập bảo mật, cấu hình tường lửa và mở rộng hệ thống tốn rất nhiều thời gian, đi ngược lại tiêu chí phát triển linh hoạt. Do đó, dự án đã quyết định sử dụng Supabase để khởi tạo nhanh chóng môi trường PostgreSQL trên điện toán đám mây (Cloud).
- Cấu trúc dữ liệu của dự án có tính liên kết quan hệ chặt chẽ (Ví dụ: Một User có thể có nhiều Review, nhiều Place nằm trong nhiều Category). Do vậy, một hệ quản trị cơ sở dữ liệu quan hệ (RDBMS) như PostgreSQL trong Supabase phù hợp hơn rất nhiều so với các giải pháp NoSQL dựa trên tài liệu (Document-based) truyền thống.

2.2.2. Quản lý xác thực (Auth) và Bảo mật cấp độ hàng (RLS)

Tính năng định danh và phân quyền là một trong những yêu cầu bắt buộc và quan trọng nhất của mọi ứng dụng. Supabase cung cấp một module Supabase Auth toàn diện, hỗ trợ quản lý đăng ký, đăng nhập an toàn bằng tiêu chuẩn mã thông báo JSON Web Token (JWT) và mã hóa mật khẩu một cách bảo mật theo chuẩn ngành.

- Bảo mật dữ liệu với Row Level Security (RLS): RLS là một tính năng bảo mật tiên tiến ở cấp độ nhân (core) của PostgreSQL mà Supabase khai thác tối đa [8]. RLS cho phép tạo ra các chính sách (policies) gắn liền trực tiếp với bảng dữ liệu thay vì viết logic kiểm tra quyền trong mã ứng dụng.

- Ứng dụng thực tế trong dự án: Dự án triển khai RLS tuân theo nguyên tắc "đặc quyền tối thiểu" (Principle of least privilege). Cụ thể:
 - + Tất cả người dùng (bao gồm khách chưa đăng nhập) đều có quyền thực thi lệnh SELECT để đọc thông tin địa điểm và bình luận.
 - + Tuy nhiên, chỉ người dùng đã sở hữu token xác thực hợp lệ mới được cấp quyền thực thi lệnh INSERT vào bảng Favorite (Lưu địa điểm yêu thích) hoặc Review (Viết đánh giá).
 - + Quan trọng nhất, chính sách RLS chặn hoàn toàn việc sửa đổi dữ liệu trái phép: Một người dùng chỉ có quyền UPDATE hoặc DELETE bản ghi đánh giá (Review) nếu giá trị user_id của bản ghi đó trùng khớp chính xác với id của họ trong JWT hiện tại. Cơ chế này bảo vệ hệ thống khỏi các lỗ hổng leo thang đặc quyền (Privilege Escalation).

2.2.3. Tương tác cơ sở dữ liệu bằng Prisma ORM

Prisma là một Trình ánh xạ quan hệ đối tượng (Object-Relational Mapper - ORM) thế hệ mới, được thiết kế chuyên biệt cho hệ sinh thái Node.js và TypeScript [3]. Trái ngược với các ORM truyền thống, Prisma tiếp cận bài toán quản trị CSDL thông qua một mô hình khai báo rõ ràng bằng tệp schema.prisma.

- Vai trò trong dự án: Prisma Client được tích hợp vào dự án để đóng vai trò là tầng truy cập dữ liệu chính, thay thế hoàn toàn việc viết các câu truy vấn SQL thô (Raw SQL) phức tạp và dễ gây lỗi.
 - + An toàn kiểu dữ liệu (Type-Safety): Khi định nghĩa cấu trúc CSDL trong tệp Schema, Prisma tự động biên dịch và tạo ra các kiểu dữ liệu tương ứng cho TypeScript. Nhờ vậy, mọi tương tác với CSDL (lọc, chèn, xóa) đều được tự động hoàn thiện mã (autocomplete) và đảm bảo cấu trúc dữ liệu luôn chính xác 100%.
 - + Ngăn chặn rủi ro bảo mật: Cú pháp của Prisma Client tự động cô lập (sanitize) các tham số đầu vào, giúp hệ thống miễn nhiễm hoàn toàn với các cuộc tấn công tiêm mã độc (SQL Injection).
 - + Đồng bộ lược đồ dữ liệu (Migrations): Tính năng Prisma Migrate cho phép đội ngũ phát triển theo dõi lịch sử thay đổi cấu trúc bảng dưới dạng các đoạn script SQL tuần tự, giúp dễ dàng triển khai (deploy) cơ sở dữ liệu lên

bắt kỳ môi trường nào (Staging, Production) một cách nhất quán và minh bạch.

2.3. Tích hợp Trí tuệ nhân tạo và Dữ liệu ngoại vi

2.3.1. Dịch vụ thời tiết OpenWeather API

Yếu tố "thông minh" cốt lõi của nền tảng dựa trên việc thu thập và phân tích được ngữ cảnh không gian và thời gian thực tế của du khách. Hệ thống tích hợp dịch vụ cung cấp dữ liệu khí tượng toàn cầu OpenWeather API [5] để truy xuất tình trạng thời tiết tại Đà Lạt theo thời gian thực.

- Vai trò trong dự án: Thông qua giao thức RESTful API, hệ thống thực hiện truy vấn đến máy chủ OpenWeather bằng tọa độ vĩ độ và kinh độ của Đà Lạt. Dữ liệu trả về (ở định dạng JSON) bao gồm nhiệt độ, độ ẩm, và các mã ID phân loại trạng thái khí tượng (ví dụ: Clear, Rain, Clouds). Backend của dự án sẽ tiến hành xử lý dữ liệu thô này, sau đó đối chiếu với các thuộc tính của các địa điểm trong cơ sở dữ liệu (đặc biệt là cờ indoorSuitable).
- Thuật toán đề xuất sẽ tự động điều chỉnh trọng số: ưu tiên đẩy lên top đầu các địa điểm trong nhà như quán cà phê, bảo tàng khi phát hiện thời tiết có mưa; và gợi ý các hoạt động trekking, cắm trại ngoài trời khi thời tiết khô ráo, nắng đẹp. Tính năng này mang lại giá trị thực tiễn vô cùng lớn, giải quyết trực tiếp pain-point của khách du lịch.

2.3.2. Nền tảng Trí tuệ nhân tạo Generative AI (Google Gemini)

Để hiện thực hóa tính năng "Trợ lý ảo" và nâng cấp trải nghiệm người dùng từ hình thức "Tra cứu thông tin tĩnh" (Tìm kiếm thụ động) sang mô hình "Tư vấn tương tác" (Chủ động đối thoại), dự án quyết định tích hợp Nền tảng Google Generative AI thông qua bộ mô hình **Gemini** thế hệ mới [4]. Các mô hình ngôn ngữ lớn (LLM) hiện đại sở hữu khả năng ấn tượng trong việc xử lý ngôn ngữ tự nhiên (NLP), duy trì ngữ cảnh hội thoại và sinh văn bản có tính logic cao.

- Vai trò trong dự án: Gemini API đóng vai trò là "bộ não" phân tích ngôn ngữ cho phân hệ Chatbot nội trú của nền tảng. Nhằm đảm bảo AI cung cấp các lời khuyên chuyên môn chính xác về du lịch Đà Lạt thay vì những kiến thức chung chung (hallucination), kỹ thuật Context-Aware Prompting (Tạo lời nhắc nhận thức ngữ cảnh) được áp dụng.

- Cụ thể, khi người dùng đặt câu hỏi, máy chủ Backend sẽ không gửi thẳng câu hỏi đó cho AI. Thay vào đó, API sẽ thực hiện bước thu thập ngữ cảnh: Lấy dữ liệu thời tiết hiện hành từ OpenWeather, truy xuất danh sách các điểm đến được đánh giá cao từ cơ sở dữ liệu Prisma, kết hợp các quy tắc hoạt động (System Instructions) được định nghĩa sẵn. Toàn bộ gói thông tin này được đóng gói thành một prompt tổng hợp và gửi đến Gemini API. Kết quả trả về là một lộ trình gợi ý hoặc một đoạn tư vấn hoàn toàn cá nhân hóa, tự nhiên, bám sát với thực tế môi trường tại Đà Lạt và đáp ứng chính xác nhu cầu của du khách.

2.4. Công cụ Đóng gói và Triển khai (DevOps)

2.4.1. Ảo hóa ứng dụng với công nghệ Docker

Docker là một nền tảng phần mềm mã nguồn mở cung cấp công nghệ ảo hóa cấp hệ điều hành, cho phép các nhà phát triển đóng gói ứng dụng, cấu hình và các thư viện phụ thuộc (dependencies) vào các khối tiêu chuẩn hóa gọi là thùng chứa (containers) [9].

- Vai trò trong dự án: Vấn đề muôn thuở trong quá trình phát triển phần mềm là sự sai lệch môi trường giữa máy tính của lập trình viên và máy chủ thực tế (It works on my machine). Dự án khắc phục triệt để rủi ro này bằng cách xây dựng Dockerfile.
- Toàn bộ mã nguồn Next.js được cấu trúc theo mô hình Multi-stage build (Xây dựng đa giai đoạn) để tách biệt môi trường cài đặt thư viện gốc (dependencies) và môi trường biên dịch mã (build). Bản dựng cuối cùng (production image) bị lược bỏ hoàn toàn các tệp mã nguồn thô và công cụ phát triển, giúp tối ưu hóa dung lượng (image size) và nâng cao bảo mật.
- Công cụ docker-compose.yml được sử dụng để định nghĩa cấu hình mạng, biến môi trường và thiết lập tham số khởi chạy tự động, giúp toàn bộ hệ thống được triển khai lên máy chủ thực tế chỉ với một câu lệnh khởi tạo duy nhất.

2.4.2. Triển khai máy chủ (VPS) và Chứng chỉ bảo mật (SSL/TLS)

Để sản phẩm thực sự đến được với người dùng cuối và đạt yêu cầu của bài đồ án, dự án không chỉ chạy trên máy tính cục bộ (localhost) mà được triển khai đưa lên môi trường mạng Internet toàn cầu thông qua kiến trúc Máy chủ riêng ảo (Virtual Private Server - VPS).

- Vai trò trong dự án: Quá trình triển khai bao gồm việc cấu hình Docker Container vận hành trực tiếp trên hệ điều hành Linux của VPS. Dự án sử dụng một tên miền (Domain) định danh rõ ràng, trỏ trực tiếp bản ghi DNS (Domain Name System) về địa chỉ IP tĩnh của máy chủ.
- Nhằm bảo vệ dữ liệu xác thực, mật khẩu của người dùng và đáp ứng tiêu chuẩn an toàn web hiện đại, hệ thống triển khai một Reverse Proxy (thường sử dụng Nginx hoặc Cloudflare Tunnel) để tích hợp chứng chỉ bảo mật SSL/TLS. Quá trình này thiết lập kênh giao tiếp mã hóa HTTPS vững chắc, đảm bảo mọi luồng dữ liệu truyền tải giữa thiết bị của du khách và hệ thống máy chủ Dalat SmartRoute AI đều được bảo vệ toàn vẹn khỏi các rủi ro đánh cắp dữ liệu hay tấn công trung gian (Man-in-the-middle attack).

CHƯƠNG 3: KIẾN TRÚC HỆ THỐNG

3.1. Phân tích và đặc tả yêu cầu hệ thống

Dựa trên bối cảnh và mục tiêu dự án, nền tảng Dalat SmartRoute AI được phân tích thành hai nhóm yêu cầu cốt lõi thông qua kỹ thuật thu thập yêu cầu (Requirement Elicitation): Yêu cầu chức năng (Functional Requirements - FR) và Yêu cầu phi chức năng (Non-functional Requirements - NFR).

3.1.1. Đặc tả yêu cầu chức năng (Functional Requirements - FR)

Hệ thống phải cung cấp đầy đủ các khối chức năng phục vụ cho 3 nhóm đối tượng chính: Khách vãng lai, Người dùng đã đăng ký và Quản trị viên. Nhằm mục đích dễ dàng theo dõi và kiểm thử (traceability), các yêu cầu chức năng được phân cụm thành từng module và đánh mã định danh (FR-XX).

Module 1: Phân hệ Xác thực và Quản lý tài khoản (Authentication & User Management)

- FR-01 (Đăng ký tài khoản): Hệ thống cho phép người dùng chưa định danh tạo tài khoản mới bằng cách cung cấp các thông tin bắt buộc: Địa chỉ Email, Tên hiển thị (Username) và Mật khẩu. Mật khẩu phải tuân thủ quy tắc độ phức tạp và được mã hóa trước khi lưu trữ xuống cơ sở dữ liệu.
- FR-02 (Đăng nhập hệ thống): Cho phép người dùng xác thực thông tin đăng nhập. Nếu hợp lệ, hệ thống cấp phát một mã thông báo (JSON Web Token - JWT) để duy trì phiên làm việc an toàn mà không cần lưu trạng thái trên máy chủ (stateless).
- FR-03 (Quản lý hồ sơ cá nhân): Người dùng đã đăng nhập có quyền truy cập trang hồ sơ cá nhân để xem, cập nhật thông tin (như Ảnh đại diện, tên hiển thị) và thực hiện đổi mật khẩu.
- FR-04 (Đăng xuất): Người dùng có thể hủy phiên làm việc hiện tại, hệ thống sẽ xóa bỏ token ở phía trình duyệt khách.

Module 2: Phân hệ Khám phá và Gợi ý địa điểm (Discovery & Recommendations)

- FR-05 (Duyệt danh sách địa điểm): Hệ thống cung cấp danh sách tổng hợp các địa điểm du lịch, không gian ẩm thực, lưu trú tại Đà Lạt. Giao diện danh sách phải hỗ trợ phân trang (Pagination) hoặc tải thêm linh hoạt (Infinite Scroll) để tối ưu hiệu năng hiển thị.

- FR-06 (Lọc và Tìm kiếm nâng cao): Cung cấp công cụ tìm kiếm địa điểm theo từ khóa (khớp với Tiêu đề và Mô tả) và bộ lọc đa chiều (Lọc theo danh mục Category, lọc theo điểm số Rating).
- FR-07 (Gợi ý theo ngữ cảnh thời tiết): Đây là tính năng nghiệp vụ cốt lõi mang lại giá trị khác biệt của nền tảng. Hệ thống tự động truy xuất dữ liệu khí hậu hiện hành tại tọa độ thành phố Đà Lạt (thông qua API OpenWeather). Dựa vào mã trạng thái thời tiết (Ví dụ: Trời đang mưa, sương mù hay nắng gắt), hệ thống tự động tinh chỉnh thuật toán để ưu tiên đề xuất các địa điểm có thuộc tính không gian phù hợp (Ví dụ: kích hoạt tham số indoorSuitable = true khi có mưa).

Module 3: Phân hệ Tương tác cộng đồng (Community Interactions)

- FR-08 (Lưu trữ danh mục yêu thích): Người dùng (đã xác thực) có quyền thêm hoặc xóa nhanh một địa điểm khỏi danh sách "Yêu thích" cá nhân, hỗ trợ việc lập kế hoạch lịch trình cá nhân.
- FR-09 (Đánh giá và Bình luận - Review): Người dùng được phép chia sẻ trải nghiệm cá nhân bằng cách viết bài đánh giá chi tiết cho một địa điểm, kèm theo việc chấm điểm dựa trên thang đo Likert (Từ 1 đến 5 sao).
- FR-10 (Tương tác bình chọn hữu ích): Cộng đồng có thể tương tác chéo bằng cách nhấn "Hữu ích" (Helpful) cho các bài đánh giá chất lượng của người khác. Hệ thống dựa vào chỉ số này để sắp xếp hiển thị các đánh giá có giá trị nhất lên đầu trang.

Module 4: Phân hệ Trợ lý ảo AI (AI Assistant)

- FR-11 (Giao diện hội thoại thông minh): Hệ thống cung cấp một Widget giao tiếp trực tuyến (Chat UI) thân thiện, hoạt động 24/7.
- FR-12 (Truy vấn và Phân tích ngữ cảnh): Trợ lý ảo tiếp nhận câu hỏi bằng ngôn ngữ tự nhiên của người dùng, tự động tổng hợp kết hợp với ngữ cảnh thực tế (Thời tiết hiện tại, danh sách địa điểm đánh giá cao nhất trong cơ sở dữ liệu) để sinh ra câu trả lời tư vấn lịch trình một cách tự nhiên, logic và mang đậm tính địa phương (local context).

Module 5: Phân hệ Quản trị hệ thống (Admin Panel)

- FR-13 (Quản lý kho dữ liệu địa điểm): Quản trị viên (Admin) được cấp quyền thực thi các thao tác CRUD (Tạo mới, Xem, Cập nhật, Xóa) đối với toàn bộ dữ liệu địa điểm du lịch trên nền tảng.

- FR-14 (Quản lý danh mục): Quản trị viên có thể thêm mới hoặc sửa đổi các Danh mục (Category) để phân loại địa điểm.
- FR-15 (Kiểm duyệt nội dung cộng đồng): Quản trị viên được cung cấp công cụ để theo dõi luồng đánh giá của người dùng và gỡ bỏ trực tiếp các bài viết vi phạm tiêu chuẩn cộng đồng, sử dụng ngôn từ không phù hợp.

3.1.2. Đặc tả yêu cầu phi chức năng (Non-functional Requirements - NFR)

Bên cạnh các tính năng hữu hình, hệ thống phải đáp ứng các tiêu chuẩn kỹ thuật khắt khe (NFR) để đảm bảo tính ổn định và trải nghiệm người dùng tối ưu.

- NFR-01 (Bảo mật và Toàn vẹn dữ liệu - Security): Toàn bộ mật khẩu của người dùng tuyệt đối không được lưu dưới dạng văn bản thô (plain-text) mà phải được băm (hashing) bằng thuật toán bcrypt với cơ chế sinh chuỗi ngẫu nhiên (salt). Mọi API giao dịch dữ liệu nhạy cảm phải được bảo vệ kép: vòng ngoài bằng Middleware kiểm tra tính hợp lệ của JWT, và vòng trong bằng cơ chế Row Level Security (RLS) của hệ quản trị CSDL PostgreSQL. Mọi truy vấn CSDL bắt buộc phải đi qua Prisma ORM để loại trừ hoàn toàn rủi ro bị tiêm nhiễm mã độc (SQL Injection).
- NFR-02 (Hiệu năng và Thời gian phản hồi - Performance): Thời gian kết xuất lần đầu (First Contentful Paint - FCP) của các trang danh sách không vượt quá 2 giây trên mạng 4G tiêu chuẩn. Hệ thống đạt được điều này nhờ kỹ thuật Server-Side Rendering (SSR). Đối với các tác vụ phụ thuộc vào độ trễ mạng ngoại vi như gọi API Gemini AI hay OpenWeather, hệ thống phải cung cấp chỉ báo trạng thái tải (Loading Skeleton hoặc Spinner) liên tục để ngăn chặn cảm giác hệ thống bị treo (unresponsive).
- NFR-03 (Tính tương thích và Khả năng sử dụng - Usability): Giao diện (UI) phải tuân thủ nghiêm ngặt triết lý thiết kế Mobile-First. Ứng dụng phải tương thích và hiển thị hoàn hảo trên mọi độ phân giải màn hình: từ điện thoại thông minh di động (chiều rộng dưới 768px), máy tính bảng (768px - 1024px) cho đến màn hình máy tính để bàn (trên 1024px).
- NFR-04 (Tính sẵn sàng và Khả năng mở rộng - Scalability): Hệ thống phần mềm được đóng gói hoàn toàn qua các thùng chứa ảo hóa (Docker Containers) độc lập. Kiến trúc này cho phép hệ thống dễ dàng được di chuyển, nâng cấp hoặc nhân

bản (scale-out) trên máy chủ khi lưu lượng truy cập gia tăng đột biến, đặc biệt vào các dịp lễ hội hoặc mùa cao điểm du lịch tại Đà Lạt.

3.2. Phân tích Tác nhân và Use Case

Quá trình mô hình hóa hệ thống (System Modeling) bắt đầu bằng việc xác định chính xác các đối tượng tương tác (Tác nhân) và các trường hợp sử dụng (Use Case) tiêu biểu mà hệ thống cung cấp.

3.2.1. Xác định Tác nhân (Actors)

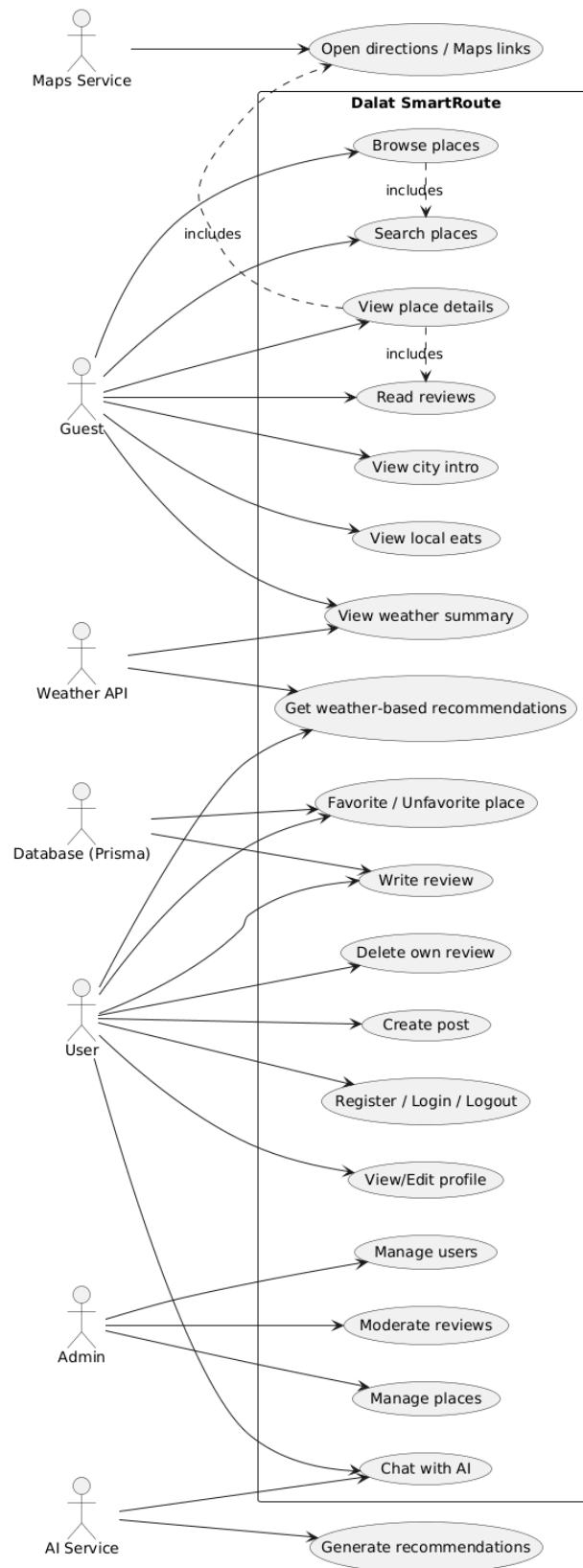
Trong hệ thống Dalat SmartRoute AI, "Tác nhân" được định nghĩa là một thực thể bên ngoài (con người hoặc hệ thống phần mềm ngoại vi khác) có thực hiện tương tác trực tiếp, trao đổi thông tin với nền tảng.

Tên Tác nhân (Actor)	Phân loại	Mô tả vai trò trong hệ thống
Guest (Khách vãng lai)	Người dùng	Du khách truy cập nền tảng để tìm kiếm thông tin nhanh chóng. Họ chưa thực hiện hành vi đăng ký hoặc đăng nhập. Quyền hạn giới hạn ở mức xem (Read-only).
User (Người dùng định danh)	Người dùng	Du khách đã thực hiện xác thực, sở hữu một không gian lưu trữ (tài khoản) riêng biệt. Được phép thực hiện các thao tác ghi dữ liệu (đánh giá, yêu thích, trò chuyện AI).
Admin (Quản trị viên)	Quản trị	Người nắm quyền điều hành cao nhất. Có đặc quyền truy cập vào các API ẩn để quản lý nội dung, thêm xóa địa điểm, và kiểm duyệt các hoạt động của nền tảng.
OpenWeather API	Hệ thống	Nguồn dữ liệu cấp phép bên ngoài (External Service) cung cấp các thông số khí hậu và đo lường thời tiết theo tọa độ địa lý (Lat, Lng) của Đà Lạt.
Google Gemini (LLM)	Hệ thống	Dịch vụ AI ngoại vi đóng vai trò tiếp nhận Prompt (lời nhắc) từ máy chủ Backend, thực thi suy luận logic và sinh ngôn ngữ tự nhiên.

Bảng 3.1: Danh sách Actors trong hệ thống

3.2.2. Biểu đồ Use Case tổng quát

Biểu đồ Use Case (Use Case Diagram) thể hiện một cái nhìn trực quan, bao quát về ranh giới của hệ thống và giới hạn quyền hạn của từng nhóm tác nhân tương tác.



Hình 3.1: Sơ đồ Use Case hệ thống

Dựa vào sơ đồ Use Case tổng quát, ta có thể thấy rõ sự phân cấp đặc quyền thông qua tính kế thừa (Generalization):

- Quản trị viên (Admin) kế thừa toàn bộ các Use Case của Người dùng (User).
 - Người dùng (User) kế thừa toàn bộ các Use Case của Khách vãng lai (Guest).
- Bên cạnh đó, mỗi quan hệ bao hàm <<includes>> được áp dụng rộng rãi để tái sử dụng nghiệp vụ. Ví dụ: Để thực thi Use Case "Viết đánh giá" (Write review) hoặc "Chat với AI", hệ thống bắt buộc phải thực thi Use Case "Xác thực" (Authenticate) trước tiên.

3.2.3. Đặc tả Kịch bản Use Case lỗi (Use Case Specifications)

Để phục vụ trực tiếp cho quá trình hiện thực hóa bằng mã nguồn, các Use Case quan trọng nhất được phân tích và đặc tả thành các kịch bản (Scenario) chi tiết, bao gồm tiền điều kiện, luồng sự kiện chính, luồng phụ và hậu điều kiện.

Kịch bản 1: Nhận Gợi ý điểm đến dựa trên thời tiết (Get weather-based recommendations)

- Tác nhân tham gia: Guest, User, OpenWeather API.
- Mô tả vắn tắt: Hệ thống tự động phân tích tình trạng thời tiết tại thời điểm truy cập và đưa ra danh sách các địa điểm phù hợp nhất để tối ưu hóa trải nghiệm của du khách.
- Tiền điều kiện: Máy chủ Backend của hệ thống hoạt động ổn định và duy trì được kết nối mạng tới OpenWeather API.
- Luồng sự kiện chính (Basic Flow):
 1. Người dùng truy cập vào phân hệ "Khám phá theo thời tiết" (Ví dụ: route /weather).
 2. Giao diện (Frontend) hiển thị trạng thái chờ (Loading) và gửi tín hiệu yêu cầu xuống Backend.
 3. Backend thiết lập HTTP Request chứa tọa độ kinh/vĩ độ của Đà Lạt và gửi tới OpenWeather API.
 4. OpenWeather API phản hồi khối dữ liệu khí tượng (Ví dụ: trạng thái Rain, nhiệt độ 18°C).
 5. Thuật toán của Backend phân tích chuỗi dữ liệu. Do nhận diện được từ khóa "Rain" (Mưa), hệ thống kích hoạt bộ lọc cơ sở dữ liệu với tiêu chí indoorSuitable = true (Không gian trong nhà).

6. Backend sử dụng Prisma ORM truy vấn CSDL PostgreSQL để lấy Top các địa điểm (Quán cafe, Bảo tàng, không gian check-in có mái che) đáp ứng tiêu chí.
 7. Backend gửi mảng dữ liệu về Frontend.
 8. Giao diện kết xuất danh sách địa điểm kèm theo thông báo thân thiện: *"Trời Đà Lạt đang mưa, đây là những góc trú mưa và thư giãn lý tưởng dành cho bạn."*
- Luồng ngoại lệ (Exception Flow): Nếu OpenWeather API không phản hồi (Timeout) hoặc hết hạn ngạch API (Rate limit), Backend sẽ bắt lỗi (Catch block) và tự động trả về một danh sách địa điểm nổi bật ngẫu nhiên (Fallback data) để đảm bảo luồng trải nghiệm không bị đứt gãy.
 - Hậu điều kiện: Người dùng nhận được danh sách địa điểm phù hợp và có thể bấm vào để xem chi tiết.

Kịch bản 2: Tương tác tư vấn với Trợ lý ảo (Chat with AI)

- Tác nhân tham gia: User, Google Gemini API.
- Mô tả vắn tắt: Người dùng trò chuyện bằng ngôn ngữ tự nhiên để yêu cầu gợi ý lịch trình, hỏi đáp thông tin du lịch và nhận lại các phản hồi được AI phân tích sâu sắc.
- Tiền điều kiện: Người dùng bắt buộc phải sở hữu phiên đăng nhập hợp lệ (có token JWT).
- Luồng sự kiện chính (Basic Flow):
 1. Người dùng mở tiện ích Chatbot (Chat Widget) ở góc màn hình và nhập câu hỏi: *"Gợi ý lịch trình 1 ngày ở Đà Lạt đi cùng gia đình có trẻ em, ưu tiên chỗ ăn uống."*
 2. Frontend gửi đoạn văn bản (Message) qua phương thức POST tới endpoint /api/chat, đính kèm JWT trong Header.
 3. Backend tiếp nhận, chạy Middleware giải mã JWT. Nếu token hợp lệ, chuyển sang bước tiếp theo.
 4. Backend tự động khởi tạo truy vấn CSDL để lấy thông tin các địa điểm có gắn nhãn phù hợp với gia đình.

5. Backend tổng hợp gói dữ liệu (Payload) bao gồm: Câu hỏi của User + Thông tin địa điểm CSDL + Chỉ thị Hệ thống (System Prompt: "Hãy đóng vai một chuyên gia du lịch Đà Lạt...").
 6. Backend mã hóa và gửi gói tin tới máy chủ Google Gemini.
 7. Gemini phân tích yêu cầu, xử lý ngôn ngữ tự nhiên và trả kết quả về Backend dưới dạng luồng dữ liệu liên tục (Readable Stream).
 8. Backend chuyển tiếp luồng Stream này về Frontend. Chữ bắt đầu xuất hiện lần lượt trên màn hình người dùng như đang gõ phím trực tiếp.
- Hậu điều kiện: Một chu kỳ hỏi-đáp hoàn tất. Lịch sử đoạn chat (Context) được lưu trữ tạm thời trên giao diện khách để làm ngữ cảnh cho câu hỏi tiếp theo.

Kịch bản 3: Thêm địa điểm vào danh sách Yêu thích (Manage Favorites)

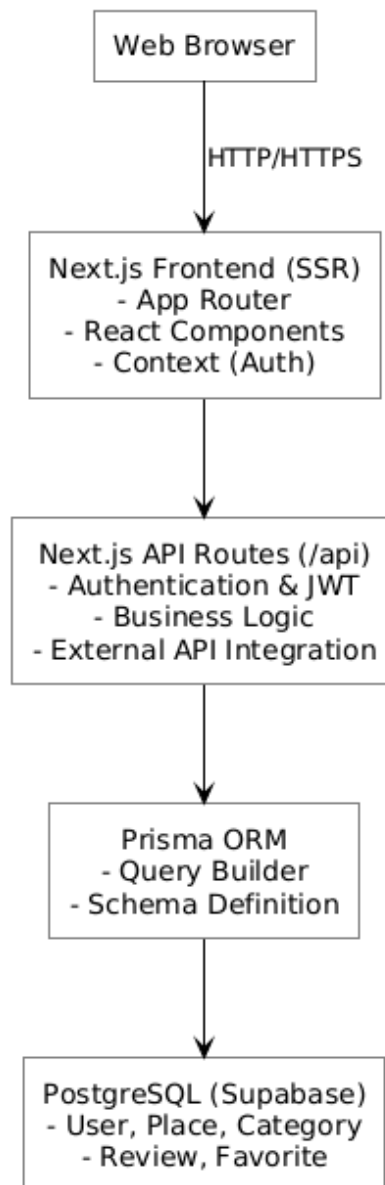
- Tác nhân tham gia: User.
- Mô tả vắn tắt: Người dùng đánh dấu lưu trữ một địa điểm quan tâm để dễ dàng truy xuất sau này.
- Tiền điều kiện: Người dùng đã xác thực (Đăng nhập). Đang đứng tại trang Chi tiết địa điểm.
- Luồng sự kiện chính (Basic Flow):
 1. Người dùng click vào biểu tượng "Trái tim" (Lưu yêu thích) trên thẻ địa điểm.
 2. Giao diện thay đổi trạng thái icon (Optimistic UI Update - Cập nhật giao diện giả định thành công để tăng độ phản hồi).
 3. Frontend gọi API POST /api/favorites với ID của địa điểm (placeId).
 4. Backend xác thực JWT, lấy ra userId.
 5. Backend truy vấn CSDL kiểm tra xem bản ghi kết hợp (userId, placeId) đã tồn tại trong bảng Favorite chưa.
 6. Do bản ghi chưa tồn tại, Backend thực thi lệnh INSERT để tạo mới.
 7. Trả về thông báo thành công (HTTP 200 OK).
- Luồng thay thế (Alternative Flow - Bỏ yêu thích): Nếu ở bước 5, Backend phát hiện bản ghi đã tồn tại (Tức là người dùng nhấn lại vào icon trái tim đang đỏ), hệ thống sẽ thực thi lệnh DELETE để gỡ bỏ địa điểm đó khỏi danh sách (Hành vi Toggle).

- Hậu điều kiện: Cơ sở dữ liệu lưu lại mối quan hệ yêu thích. Giao diện Đồng bộ hóa chính xác trạng thái của biểu tượng Trái tim.

3.3. Thiết kế Kiến trúc tổng thể hệ thống

Để giải quyết bài toán phức tạp về sự tích hợp đa dạng các dịch vụ (API thời tiết, AI, CSDL) đồng thời đáp ứng khắt khe các Yêu cầu Phi chức năng (NFR) về hiệu năng và khả năng bảo trì, hệ thống từ bỏ kiến trúc nguyên khối (Monolith) cổ điển. Thay vào đó, hệ thống áp dụng Kiến trúc đa tầng (N-Tier Architecture) được tối ưu hóa vận hành theo cơ chế BFF (Backend-for-Frontend) – một mô hình thiết kế hiện đại được hỗ trợ mặc định bên trong kiến trúc App Router của Next.js.

System Architecture Diagram



Hình 3.2: Sơ đồ kiến trúc hệ thống

Dựa vào sơ đồ trên, kiến trúc hệ thống được phân rã thành 4 lớp (Layers) logic riêng biệt. Mỗi lớp mang một phạm vi trách nhiệm duy nhất (Single Responsibility Principle) và chỉ được phép giao tiếp với lớp liền kề thông qua các giao diện (Interfaces) chuẩn mực.

3.3.1. Lớp Trình bày (Presentation Layer/Client Interface)

Vận hành trực tiếp trên môi trường trình duyệt (Web Browser) của người dùng cuối, đóng vai trò tiếp nhận các tương tác vật lý (Click, Cuộn, Gõ phím) và hiển thị dữ liệu trực quan. Tầng này khai thác triệt để sức mạnh của thư viện React.js kết hợp Next.js App Router. Thay vì tải toàn bộ mã nguồn về máy khách, hệ thống áp dụng mô hình lai:

- + React Server Components (RSC): Xử lý kết xuất nội dung tĩnh (như trang bài viết chi tiết, danh mục) ngay trên máy chủ để tăng tốc độ render HTML ban đầu, thân thiện với SEO.
- + Client Components: Xử lý các logic phía trình duyệt (Quản lý trạng thái bằng React Hooks, Context API) ở những phần tử cần tương tác (như Thanh tìm kiếm, Cửa sổ Chat). Giao diện được tạo kiểu linh hoạt bằng Tailwind CSS đảm bảo chuẩn Responsive.

3.3.2. Lớp Logic Ứng dụng và Dịch vụ (Application & Service Layer)

Được triển khai dưới dạng Next.js API Routes (Route Handlers) nằm trong thư mục /api/. Đây chính là tầng BFF (Backend-for-Frontend), đóng vai trò như một bức tường lửa logic an toàn. Trình duyệt client không bao giờ được phép gửi câu lệnh trực tiếp xuống cơ sở dữ liệu.

Mọi hành động của người dùng (từ lấy danh sách, đến bình luận) đều phải truyền tải qua các HTTP request (GET, POST, PUT, DELETE) tới tầng này. Tại đây, chuỗi luồng xử lý (Pipeline) diễn ra bao gồm: bóc tách JWT từ Header, chạy Middleware xác minh quyền hạn (Authorization), tiền xử lý và xác thực tham số đầu vào (Input validation). Tầng này cũng đóng vai trò là một Máy chủ Đại diện (Proxy) để khởi tạo các kết nối an toàn tới các Dịch vụ ngoại vi (Gemini AI, OpenWeather), đảm bảo các Khóa bảo mật API (API Keys) không bao giờ bị lộ ra ngoài trình duyệt khách.

3.3.3. Lớp Truy cập Dữ liệu (Data Access Layer - DAL)

Để duy trì tính trừu tượng, kiến trúc hệ thống không nhúng trực tiếp mã nguồn SQL thô vào tầng Logic. Thay vào đó, lớp DAL sử dụng Prisma ORM. Prisma Client

cung cấp một bộ API nội bộ an toàn với cơ chế kiểm tra kiểu nghiêm ngặt (Type-Safety) của TypeScript. Nó biên dịch các hàm (ví dụ như `prisma.place.findMany()`) thành các câu lệnh truy vấn PostgreSQL nguyên thủy được tối ưu hóa.

Lớp này đóng vai trò như một màng lọc, tự động vô hiệu hóa các ký tự độc hại, ngăn chặn hoàn toàn tấn công SQL Injection.

3.3.4. Lớp Hạ tầng Cơ sở dữ liệu (Database / Persistence Layer)

Nằm ở tầng sâu nhất của kiến trúc, đây là nơi toàn bộ dữ liệu vật lý được lưu trữ bền vững (Persistent Storage). Dự án vận hành trên dịch vụ đám mây cơ sở dữ liệu quan hệ PostgreSQL của nền tảng Supabase.

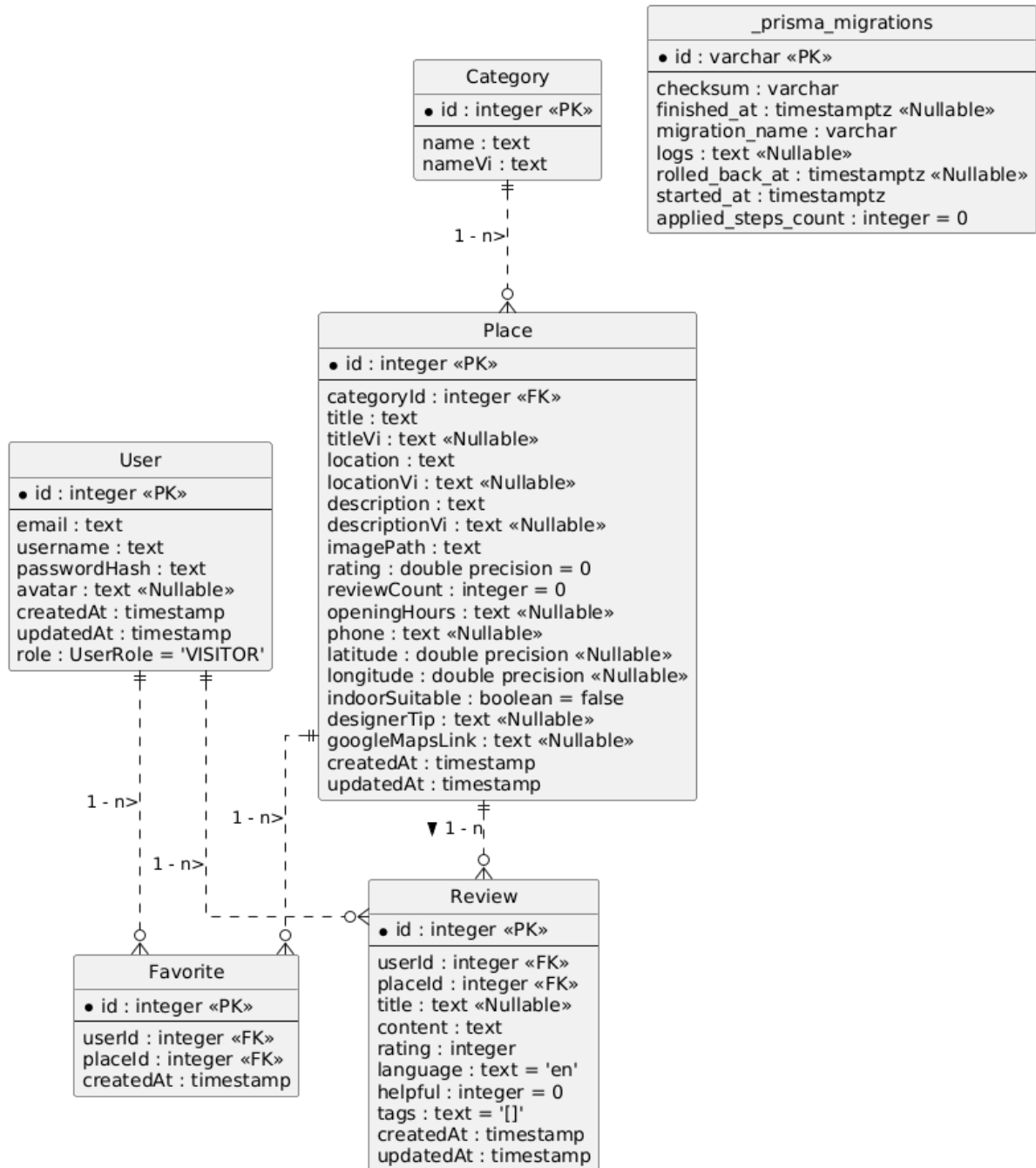
Tại tầng này, các quy tắc bảo vệ cấp độ thấp nhất là Row Level Security (RLS) được thiết lập. RLS đảm bảo rằng ngay cả khi có một lỗ hổng logic nào đó lọt qua được tầng Application, người dùng A vẫn không thể dùng mã lệnh để sửa đổi, can thiệp vào bài đánh giá (Review) của người dùng B, vì hệ quản trị CSDL sẽ chủ động từ chối giao dịch ở cấp độ hàng dữ liệu (Row-level).

3.4. Mô hình hóa và Thiết kế Cơ sở dữ liệu (ERD)

Cơ sở dữ liệu của dự án Dalat SmartRoute AI được thiết kế và tối ưu hóa đặc biệt cho cấu trúc Read-heavy (Tần suất tác vụ Đọc/Truy vấn cao gấp nhiều lần tác vụ Ghi/Nhập liệu) - đặc thù kinh điển của các nền tảng cổng thông tin du lịch. Mô hình dữ liệu tuân thủ nghiêm ngặt các quy tắc chuẩn hóa (Normalization) lên tới Dạng chuẩn 3 (3NF - Third Normal Form) nhằm đảm bảo loại bỏ hoàn toàn các hiện tượng bất thường khi cập nhật (Update Anomalies) và sự dư thừa dữ liệu (Data Redundancy).

Dựa trên Lược đồ Quan hệ Thực thể (ERD), cấu trúc CSDL được phân rã thành các bảng (Tables) độc lập nhưng có mối liên kết logic chặt chẽ. Để phục vụ việc triển khai mã nguồn Prisma Schema, dưới đây là Từ điển Dữ liệu (Data Dictionary) phân tích chi tiết từng thực thể cốt lõi.

Sơ đồ Cơ sở dữ liệu (ERD) - Dalat SmartRoute AI



Hình 3.3: Lược đồ quan hệ thực thể (ERD)

3.4.1. Thiết kế bảng dữ liệu (Data Dictionary)

3.4.1.1. Bảng User (Thông tin tài khoản người dùng)

Bảng này đảm nhiệm vai trò nền tảng cho việc quản lý định danh. Mật khẩu không bao giờ được lưu trữ dưới dạng văn bản thô (plain-text) để đề phòng sự cố rò rỉ cơ sở dữ liệu (Data breach).

Tên trường (Column)	Kiểu dữ liệu (Type)	Ràng buộc (Constraints)	Mô tả (Description)
id	Integer	Primary Key, Auto Increment	Mã định danh duy nhất của mỗi người dùng.
email	String	Unique, Not Null	Địa chỉ email dùng để đăng nhập. Phải là duy nhất.
username	String	Unique, Not Null	Tên hiển thị (Nickname) công khai trên cộng đồng.
passwordHash	String	Not Null	Chuỗi mã hóa (Hash) mật khẩu bằng thuật toán bcrypt.
avatar	String	Nullable	Đường dẫn URL tới hình ảnh đại diện của người dùng.
role	Enum	Default: 'VISITOR'	Phân quyền hệ thống (Gồm 2 loại: VISITOR hoặc ADMIN).
createdAt	DateTime	Default: now()	Nhãn thời gian (Timestamp) ghi nhận lúc khởi tạo tài khoản.

Bảng 3.2: Thiết kế bảng Thông tin tài khoản người dùng

3.4.1.2. Bảng Category (Danh mục phân loại)

Lưu trữ các nhóm phân loại chính yếu (Ví dụ: Ẩm thực, Quán Cafe, Điểm Tham Quan, Lưu trú).

Tên trường (Column)	Kiểu dữ liệu (Type)	Ràng buộc (Constraints)	Mô tả (Description)
id	Integer	Primary Key, Auto Increment	Mã định danh danh mục.
name	String	Not Null	Tên danh mục (Ngôn ngữ chuẩn - Tiếng Anh).
nameVi	String	Nullable	Tên danh mục được bản địa hóa (Tiếng Việt).

Bảng 3.3: Thiết kế bảng Danh mục phân loại

3.4.1.3. Bảng Place (Điểm đến du lịch)

Thực thể trung tâm, sở hữu khối lượng trường dữ liệu lớn và phức tạp nhất để phục vụ cho các thuật toán tìm kiếm và xây dựng ngữ cảnh LLM. Bảng được thiết kế với cơ chế phi chuẩn hóa (Denormalization) có chủ đích ở trường rating và reviewCount để tăng tốc độ tải trang.

Tên trường (Column)	Kiểu dữ liệu (Type)	Ràng buộc (Constraints)	Mô tả (Description)
id	Integer	Primary Key, Auto Increment	Mã định danh địa điểm.
categoryId	Integer	Foreign Key	Khóa ngoại liên kết tới bảng Category.
title	String	Not Null	Tên địa điểm (Ngôn ngữ mặc định/Tiếng Anh).
titleVi	String	Nullable	Tên địa điểm (Tiếng Việt) phục vụ i18n.
description	Text	Not Null	Bài viết mô tả chi tiết, hấp dẫn về địa điểm.
imagePath	String	Not Null	Đường dẫn tĩnh (URL) tới tệp hình ảnh minh họa (Thumbnail).
rating	Float	Default: 0	Tổng điểm đánh giá trung bình. Lưu trữ sẵn để tối ưu truy vấn.
reviewCount	Integer	Default: 0	Tổng số lượt bình luận đã nhận được.
latitude	Float	Nullable	Vĩ độ địa lý (Phục vụ tích hợp bản đồ và tính toán khoảng cách).
longitude	Float	Nullable	Kinh độ địa lý.
indoorSuitable	Boolean	Default: false	Cờ logic (Flag) cốt lõi: Đánh dấu địa điểm phù hợp phục vụ lúc trời mưa.

googleMapsLink	String	Nullable	Liên kết chuyển hướng (Deep link) tới ứng dụng Google Maps.
----------------	--------	----------	---

Bảng 3.4: Thiết kế bảng Điểm đến du lịch

3.4.1.4. Bảng Review (Đánh giá của cộng đồng)

Bảng giao dịch (Transaction Table) sinh ra từ tương tác của người dùng với địa điểm, đại diện cho thực thể trung gian giải quyết mối quan hệ Nhiều-Nhiều (N-N).

Tên trường (Column)	Kiểu dữ liệu (Type)	Ràng buộc (Constraints)	Mô tả (Description)
id	Integer	Primary Key, Auto Increment	Mã định danh duy nhất của bài đánh giá.
userId	Integer	Foreign Key	Khóa ngoại trở về tác giả bài viết (User).
placeId	Integer	Foreign Key	Khóa ngoại trở về địa điểm được đánh giá (Place).
content	Text	Not Null	Nội dung phản hồi, nhận xét chi tiết của du khách.
rating	Integer	Check (1-5), Not Null	Điểm số đánh giá độ hài lòng (Likert Scale).
helpful	Integer	Default: 0	Chỉ số đo lường số lượt người dùng khác đánh dấu bài viết là "Hữu ích".
createdAt	DateTime	Default: now()	Thời gian khởi tạo bài đánh giá.

Bảng 3.5: Thiết kế bảng Đánh giá của cộng đồng

3.4.1.5. Bảng Favorite (Lưu trữ Yêu thích)

Lưu vết các địa điểm mà một người dùng đánh dấu quan tâm.

Tên trường (Column)	Kiểu dữ liệu (Type)	Ràng buộc (Constraints)	Mô tả (Description)
id	Integer	Primary Key, Auto Increment	Mã định danh bản ghi yêu thích.

userId	Integer	Foreign Key	Khóa ngoại trở về Người dùng (User).
placeId	Integer	Foreign Key	Khóa ngoại trở về Địa điểm (Place).
createdAt	DateTime	Default: now()	Thời gian bấm nút lưu yêu thích.

Bảng 3.6: Thiết kế bảng Lưu trữ Yêu thích

3.4.2. Thiết lập ràng buộc Toàn vẹn tham chiếu (Referential Integrity)

Mô hình dữ liệu được thiết lập các mối quan hệ (Relationships) chặt chẽ nhằm bảo vệ cơ sở dữ liệu khỏi các dị thường.

- Quan hệ Một-Nhiều (1:N) giữa Category và Place: Một Danh mục (Ví dụ: Lưu trú) có thể tham chiếu tới hàng trăm Địa điểm. Khóa ngoại categoryId trong bảng Place đảm bảo hệ thống luôn có thể nhóm các địa điểm theo ngữ cảnh.
- Quan hệ Một-Nhiều (1:N) cấp độ tương tác tài khoản: Một Người dùng (User) có khả năng khởi tạo vô số Đánh giá (Review) và lưu trữ không giới hạn các điểm đến Yêu thích (Favorite).
- Hành vi Xóa Xếp Tầng (Cascade Delete): Để đảm bảo không xuất hiện "Dữ liệu mồ côi" (Orphan records) gây lãng phí bộ nhớ và lỗi truy vấn, Prisma ORM được cấu hình với chỉ thị onDelete: Cascade tại các khóa ngoại. Chẳng hạn, khi Quản trị viên thực thi lệnh Xóa một Place khỏi hệ thống (ví dụ do quán đã đóng cửa), hệ quản trị CSDL PostgreSQL sẽ tự động rà quét và quét sạch toàn bộ Review cũng như bản ghi Favorite đang liên kết tới địa điểm đó một cách đồng bộ và an toàn tuyệt đối ở tầng nhân (Core level).

3.5. Thiết kế Hệ thống Giao diện lập trình ứng dụng (API Specifications)

Để giao tiếp giữa Tầng Client và Server theo nguyên lý kiến trúc BFF, dự án xây dựng một tập hợp các Giao diện lập trình ứng dụng tuân thủ tiêu chuẩn RESTful API. Việc thiết kế tuân thủ tiêu chuẩn REST (sử dụng đúng các HTTP Methods như GET, POST, PUT, DELETE) giúp cấu trúc mã nguồn dự đoán được và dễ dàng bảo trì. Dưới đây là đặc tả kỹ thuật cho một số API Endpoint quan trọng nhất:

3.5.1. Endpoint Xác thực và Cấp phát Token (Login)

- URL: /api/auth/login

- Method: POST
- Mô tả: Tiếp nhận thông tin đăng nhập, đối chiếu CSDL và cấp phát JWT.
- Payload đầu vào (Request Body - JSON):


```
{
    "email": "traveler@example.com",
    "password": "hashed_secret_password"
}
```
- Phản hồi thành công (Response - 200 OK):


```
{
    "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpX...",
    "user": { "id": 101, "username": "Traveler", "role": "VISITOR" }
}
```
- Phản hồi thất bại (Response - 401 Unauthorized): Trả về khi sai mật khẩu hoặc tài khoản không tồn tại.

3.5.2. Endpoint Khai thác Địa điểm theo Thời tiết (*Weather Recommendations*)

- URL: /api/places/weather-recommendations
- Method: GET
- Query Parameters: ?weather=rainy hoặc ?weather=sunny
- Mô tả: Endpoint nghiệp vụ đặc thù. Nó tiếp nhận trạng thái thời tiết từ Client (sau khi Client đã query OpenWeather).
- Logic xử lý nội bộ: Nếu phát hiện parameter weather=rainy, API sử dụng Prisma để lọc toàn bộ các Place có điều kiện indoorSuitable: true, sau đó kết hợp thao tác ORDER BY rating DESC để trả về các địa điểm tránh mưa có chất lượng tốt nhất.
- Phản hồi thành công (Response - 200 OK): Mảng (Array) các Object chứa thông tin địa điểm (ID, Title, ImagePath).

3.5.3. Endpoint Trò chuyện với Trợ lý AI (*Generative AI Chat*)

- URL: /api/chat
- Method: POST
- Headers bắt buộc: Authorization: Bearer <JWT_Token> (Chỉ phục vụ user đã đăng nhập để kiểm soát chi phí gọi AI).
- Mô tả: Đóng vai trò là cầu nối trung gian giao tiếp với mô hình Google Gemini.

- Payload đầu vào:


```
{
  "message": "Tôi muốn đi ăn tối ở một không gian lãng mạn tại Đà Lạt"
}
```
- Dữ liệu trả về (Stream Response): Nhằm tối ưu trải nghiệm người dùng, API này không sử dụng kết nối HTTP Request/Response truyền thống (chờ AI xử lý xong 100% rồi mới trả về). Nó sử dụng chuẩn Readable Stream (Dữ liệu luồng). Các token văn bản do Gemini sinh ra sẽ được đẩy liên tục (chunk by chunk) về trình duyệt, tạo hiệu ứng chữ hiện ra từ từ như đang trò chuyện thực tế.

3.6. Thiết kế các luồng hoạt động nghiệp vụ chi tiết (Sequence Flows)

Việc biểu diễn các luồng trình tự (Sequence Diagrams/Flows) là công đoạn quan trọng nhất trong thiết kế phần mềm để làm rõ dòng thời gian (timeline), thứ tự thực thi và các bước trao đổi thông tin cụ thể giữa các thực thể hệ thống.

3.6.1. Luồng xử lý phân tích ngữ cảnh của Trợ lý AI (Google Gemini Flow)

Đây là phân hệ tinh hoa và phức tạp nhất của đồ án, đòi hỏi sự phối hợp nhịp nhàng của toàn bộ các tầng kiến trúc. Phân hệ này áp dụng kỹ thuật RAG (Retrieval-Augmented Generation - Sinh tạo tăng cường truy xuất) kết hợp Context-Aware Prompting (Lời nhắc nhận thức ngữ cảnh).

1. Khởi phát luồng (Trigger): Người dùng nhập một yêu cầu (Ví dụ: "*Trời đang mưa thì nên đi đâu uống cafe đẹp?*") vào cửa sổ Chat Widget trên giao diện người dùng.
2. Xác thực phiên (Authentication Check): Frontend tiến hành thu thập nội dung tin nhắn, đóng gói gửi lên /api/chat kèm theo chuỗi bảo mật JWT ở phần Header. Tại máy chủ Backend, một Middleware được thực thi để giải mã JWT. Nếu token hết hạn (Expired) hoặc bị giả mạo (Invalid Signature), hệ thống trả về mã lỗi HTTP 401 Unauthorized, lập tức đóng luồng để tiết kiệm tài nguyên.
3. Xây dựng Ngữ cảnh Nhận thức (Context Building): Tại bước mang tính quyết định này, Backend KHÔNG gửi ngay câu hỏi thô (raw prompt) lên Google LLM mà tiến hành nạp "tri thức nền" (Background knowledge):
 - + Bước 3a - Ngữ cảnh Không gian: Backend gọi tiện ích nội bộ truy xuất tọa độ và gửi yêu cầu GET ẩn danh tới OpenWeather API để lấy báo cáo tình trạng khí hậu hiện tại (Ví dụ: Trạng thái mưa, nhiệt độ 16 độ).

- + Bước 3b - Ngữ cảnh Dữ liệu chuyên ngành: Backend điều khiển Prisma ORM truy xuất bảng Place, lấy danh sách Top 5 quán cafe (Category) có cờ indoorSuitable = true (Phù hợp trú mưa) và có điểm rating cao nhất từ CSDL PostgreSQL.
 - + Bước 3c - Chỉ thị Hệ thống (System Instructions): Backend nạp các quy tắc ứng xử (Persona). Hệ thống thiết lập AI phải nhập vai là *"Một hướng dẫn viên địa phương Đà Lạt thân thiện, đưa ra câu trả lời súc tích, chỉ sử dụng thông tin từ danh sách địa điểm được cung cấp"*.
4. **Hợp nhất và Truyền tải (Injection & Transport):** Toàn bộ khối thông tin đồ sộ này (Ngữ cảnh Thời tiết + Danh sách địa điểm + Chỉ thị hệ thống + Câu hỏi nguyên bản của người dùng) được lắp ghép thành một Payload khổng lồ (Context-Rich Prompt). Gói tin này được gửi qua kênh mã hóa HTTPS tới máy chủ Google Generative AI API.
 5. **Khai thác phản hồi luồng (Streaming Response):** Siêu máy tính của mô hình Gemini tiến hành phân tích chuỗi dữ liệu đầu vào, loại bỏ các thông tin nhiễu, và tổng hợp ngôn ngữ tự nhiên để sinh ra một đoạn tư vấn. Thay vì gửi một cục dữ liệu lớn, Gemini trả về các mảnh (chunks). Backend Node.js chuyển tiếp luồng Stream này thẳng về Frontend. Giao diện Chat UI bắt các mảnh này và hiển thị mượt mà trên màn hình người dùng, kết thúc một chu kỳ hội thoại khép kín, thông minh và siêu cá nhân hóa.

3.6.2. Luồng giao dịch tạo lập và cập nhật Đánh giá cộng đồng (Review System)

Hệ thống đánh giá là điểm nhấn thể hiện mục tiêu "Community-driven" của dự án. Một luồng nghiệp vụ tạo Review không đơn giản chỉ là thêm một dòng chữ vào CSDL, mà nó được thiết kế chặt chẽ như một giao dịch đồng thời (Database Transaction) để đảm bảo tính nhất quán dữ liệu.

1. **Khởi phát:** Người dùng (sau khi đã trải nghiệm) nhấn nút "Viết Đánh giá" trên trang chi tiết của một địa điểm (Place Detail Page).
2. **Kiểm tra trạng thái xác thực phía Client:** Client Component React tiến hành kiểm tra trạng thái lưu trong Context API. Nếu biến isAuthenticated là false, hệ thống chặn sự kiện, mở Modal (Hộp thoại) yêu cầu người dùng phải thực hiện thao tác Đăng nhập trước khi tiếp tục.

3. Thu thập nội dung: Khi đã đăng nhập, người dùng hoàn thiện biểu mẫu giao diện (Form) bao gồm: Click số điểm đánh giá (Từ 1-5 sao), nhập Tiêu đề bài viết và Nội dung phản hồi chi tiết. Nút Submit (Gửi đi) được kích hoạt.
4. Giao tiếp API và Định danh: * Frontend phát đi một phương thức POST /api/reviews với Payload body chứa: placeId, rating, content.
 - + Ở phía Backend, Tầng Application bóc tách userId từ JWT (đã được xác thực qua Middleware).
 - + Việc lấy userId từ Server thay vì Client truyền lên giúp đảm bảo tính chính danh, ngăn chặn việc người dùng dùng công cụ hacker giả mạo định danh của người khác để bình luận xấu.
5. Thực thi Giao dịch Đồng thời (Database Transaction & Denormalization): Để tăng tốc độ hiển thị cho hệ thống, dự án áp dụng chiến lược phi chuẩn hóa (Denormalization) số lượng Review và Rating ở bảng Place. Do đó, việc lưu một đánh giá cần thực hiện liên hoàn 2 thao tác:
 - + Thao tác 1: Prisma thực thi lệnh INSERT vào bảng Review với các dữ liệu vừa thu thập.
 - + Thao tác 2: Ngay khi INSERT thành công, một thủ tục tính toán tiếp tục được kích hoạt để gọi lệnh UPDATE vào bảng Place. Hệ thống truy vấn CSDL để lấy tổng số điểm hiện có của địa điểm, cộng với điểm số mới, tính ra trung bình cộng mới. Đồng thời, biến reviewCount được cập nhật: $reviewCount = reviewCount + 1$. *(Quá trình này lý tưởng nhất được thực thi trong một Prisma.\$transaction để đảm bảo: Nếu có lỗi ở Thao tác 2 thì Thao tác 1 cũng tự động bị hủy (Rollback), đảm bảo tính toàn vẹn cơ sở dữ liệu).*
6. Hoàn tất luồng: API phản hồi HTTP mã 201 Created về cho trình duyệt. Frontend bắt được sự kiện thành công, tiến hành xóa rỗng form (reset) và sử dụng cơ chế Soft Navigation của Next.js (như router.refresh()) để cập nhật ngay lập tức bài đánh giá vừa viết lên giao diện mà không cần người dùng phải tự tải lại (F5) trang web.

3.7. Thiết kế Giao diện người dùng (UI/UX)

Yếu tố cuối cùng cấu thành nên sự thành công của giai đoạn thiết kế là giao diện tương tác (UI/UX). Đối với một nền tảng khám phá du lịch, thiết kế thẩm mỹ và luồng thao tác đóng vai trò quyết định trong việc giữ chân du khách.

- **Triết lý Mobile-First:** Do thói quen tìm kiếm thông tin du lịch chủ yếu qua điện thoại thông minh, toàn bộ giao diện của Dalat SmartRoute AI được thiết kế bắt đầu từ khung nhìn di động (Mobile viewport), sau đó mới tinh chỉnh (scale-up) cho màn hình lớn (Tablet, Desktop).
- **Hệ thống Design System với Tailwind CSS:** Dự án duy trì một sự nhất quán cao độ về khoảng cách (margin/padding), kiểu chữ (typography) và bảng màu (color palette). Tone màu chủ đạo được định hướng phản ánh không khí của Đà Lạt (Tươi mát, hiện đại). Các lớp tiện ích của Tailwind như rounded-xl, shadow-lg được sử dụng để tạo độ bo góc mềm mại và độ nổi (depth) cho các thẻ địa điểm (Place Cards), mang lại cảm giác thiết kế ứng dụng cao cấp (premium).
- **Trải nghiệm mượt mà (Micro-interactions):** Mọi thao tác chờ dữ liệu từ máy chủ (chuyển trang, nạp đánh giá, hỏi AI) đều được trang bị các hiệu ứng Skeleton Loading hoặc Spinner. Việc xử lý tối ưu các trạng thái chờ này giúp xóa bỏ cảm giác độ trễ hệ thống, nâng cao sự hài lòng (Satisfaction) của người dùng cuối.

CHƯƠNG 4: XÂY DỰNG VÀ TRIỂN KHAI HỆ THỐNG

4.1. Tổ chức cấu trúc mã nguồn (Source Code Organization)

Thay vì thiết lập các kho lưu trữ rời rạc, dự án áp dụng mô hình **Monorepo** nhằm quản lý tập trung cả giao diện người dùng (Frontend) và hệ thống dịch vụ API (Backend) trong cùng một hệ sinh thái. Cấu trúc mã nguồn được phân rã theo nguyên tắc Phân tách trách nhiệm (Separation of Concerns), chia thành các phân hệ logic rõ ràng nhằm đảm bảo khả năng mở rộng và dễ dàng bảo trì.

4.1.1. Phân hệ Giao diện và Định tuyến (Frontend & Routing)

Nằm toàn bộ trong thư mục `src/app/`, tuân thủ triết lý của kiến trúc Next.js App Router.

- Nhóm tuyến đường hiển thị (Pages): Các thư mục con như `/(pages)/admin/`, `/(pages)/place/[id]/` đại diện cho các tuyến đường (routes) tĩnh và động trên giao diện. Tại đây, các tệp `page.tsx` chịu trách nhiệm kết xuất dữ liệu HTML phía máy chủ (SSR) để tối ưu SEO.
- Nhóm thành phần giao diện (Components & Views): Thư mục `src/components/` lưu trữ các khối giao diện có khả năng tái sử dụng cao (như Navbar, Footer, PlaceCard, ChatWidget). Trong khi đó, thư mục `src/views/` chứa các tổ hợp giao diện phức tạp cấp độ trang (Page-level blocks). Việc tách biệt này giúp mã nguồn của các tệp Route trở nên gọn nhẹ và dễ đọc.

4.1.2. Phân hệ Dịch vụ Backend (API Layer)

Được cô lập hoàn toàn tại `src/app/api/`. Đây là nơi chứa các Route Handlers đóng vai trò như các RESTful Endpoints.

- Các thư mục con như `/auth`, `/places`, `/weather`, `/chat` phản ánh đúng các miền nghiệp vụ (Domain Logic) của hệ thống.
- Kiến trúc này ngăn chặn tuyệt đối rủi ro mã thực thi máy chủ (Server-side logic) hoặc các khóa bí mật (API Keys) bị rò rỉ vào các tệp tin gửi xuống trình duyệt khách.

4.1.3. Phân hệ Tương tác Dữ liệu (Database ORM)

Khu vực `prisma/` là trái tim của tầng lưu trữ. Tệp `schema.prisma` đóng vai trò là "Bản thiết kế duy nhất" (Single Source of Truth) định nghĩa toàn bộ mô hình thực thể. Thư mục `migrations/` lưu giữ lịch sử các thay đổi cấu trúc bảng dưới dạng mã SQL, giúp

đồng bộ hóa CSDL giữa môi trường cục bộ (Local) và máy chủ (Production) một cách an toàn.

4.2. Triển khai Cơ sở dữ liệu trên môi trường Supabase

Để hệ thống có thể vận hành trực tuyến, bước đầu tiên và quan trọng nhất là triển khai cơ sở dữ liệu lên dịch vụ đám mây (Cloud Database). Dự án sử dụng nền tảng Supabase, vận hành lõi PostgreSQL. Quá trình triển khai đòi hỏi các cấu hình kỹ thuật chuyên sâu để giải quyết bài toán "Cạn kiệt kết nối" (Connection Exhaustion) thường gặp trong các kiến trúc Serverless.

4.2.1. Cấu hình Chuỗi kết nối (Connection Pooling)

Khi ứng dụng Next.js triển khai trên môi trường Serverless (như Vercel), mỗi yêu cầu (Request) có thể khởi tạo một môi trường thực thi (Function) mới, dẫn đến việc mở hàng ngàn kết nối đồng thời tới CSDL và làm sập máy chủ (Crash). Để giải quyết, dự án cấu hình hai chuỗi kết nối riêng biệt trong tệp .env:

- DATABASE_URL (Transaction Pooling): Sử dụng Supavisor/PgBouncer do Supabase cung cấp (thường chạy trên cổng 6543). Chuỗi kết nối này được Prisma Client sử dụng trong thời gian chạy (Runtime). Nó giúp gom nhóm và tái sử dụng các kết nối CSDL, đảm bảo hiệu năng cao khi có hàng ngàn du khách truy cập cùng lúc.
- DIRECT_URL (Session Connection): Là chuỗi kết nối trực tiếp vào cổng 5432 của PostgreSQL. Chuỗi này chỉ được Prisma sử dụng khi thực thi các lệnh thay đổi cấu trúc bảng (Migrations) nhằm đảm bảo giao dịch không bị ngắt quãng bởi bộ đệm Pooling.

4.2.2. Triển khai Lược đồ và Nạp dữ liệu mẫu (Migration & Seeding)

Quá trình đưa thiết kế CSDL từ máy tính cục bộ lên Supabase được tự động hóa qua các công cụ của Prisma:

1. Apply Migrations: Quản trị viên thực thi lệnh `npx prisma migrate deploy`. Hệ thống sẽ đối chiếu lịch sử trong thư mục `prisma/migrations` và chạy các kịch bản SQL lên Supabase để tạo lập các bảng User, Place, Review cùng các khóa ngoại tương ứng một cách đồng bộ.
2. Khởi tạo dữ liệu (Seeding): Để nền tảng Dalat SmartRoute AI có sẵn dữ liệu hoạt động ngay lập tức, tập lệnh `prisma/seed.js` được thực thi (`npx prisma db seed`). Lệnh này tự động tiêm (inject) các tài khoản Admin mặc định, thiết lập danh mục

(Categories) và hàng chục địa điểm du lịch tiêu biểu tại Đà Lạt vào CSDL Supabase.

4.3. Đóng gói ứng dụng với Docker (Containerization)

Đề tuân thủ các quy chuẩn khắt khe về kỹ nghệ phần mềm và đáp ứng tính di động (Portability) của dự án, toàn bộ ứng dụng được ảo hóa (Dockerized). Quá trình này giúp loại bỏ triệt để hiện tượng "Ứng dụng chạy tốt trên máy phát triển nhưng lỗi trên máy chủ" do sai khác môi trường hệ điều hành.

4.3.1. Tối ưu hóa với Dockerfile (Multi-stage Build)

Để tối ưu hóa dung lượng của gói Container cuối cùng và tăng cường độ bảo mật, tệp Dockerfile của dự án không sử dụng phương pháp đóng gói đơn lớp truyền thống mà áp dụng kỹ thuật **Multi-stage Build** (Xây dựng đa giai đoạn) chia làm 3 bước cốt lõi:

1. Giai đoạn deps (Cài đặt phụ thuộc)

- + Sử dụng hệ điều hành Alpine Linux thu gọn (node:18-alpine).
- + Chỉ sao chép tệp package.json và package-lock.json để tiến hành cài đặt toàn bộ các thư viện. Việc này giúp tận dụng bộ nhớ đệm (Cache) của Docker, tăng tốc độ Build cho các lần sau.

2. Giai đoạn builder (Biên dịch mã nguồn)

- + Chạy lệnh `npx prisma generate` để tạo ra bộ thư viện Prisma Client nội bộ, ánh xạ chính xác với cấu trúc CSDL trên Supabase.
- + Kích hoạt lệnh `npm run build`. Next.js tiến hành tối ưu hóa tài nguyên tĩnh, biên dịch TypeScript thành JavaScript chuẩn, và kết xuất trước (Pre-render) HTML. Chế độ standalone được bật trong cấu hình Next.js để thu gom tự động các tệp tin cần thiết nhất.

3. Giai đoạn runner (Môi trường vận hành)

- + Đây là lớp Container cuối cùng sẽ được đem đi triển khai (Deploy). Lớp này chỉ sao chép các tệp tin đã được biên dịch thành công từ giai đoạn builder (bao gồm thư mục `.next/standalone`).
- + Loại bỏ hoàn toàn mã nguồn thô và các công cụ lập trình, giúp giảm dung lượng Image từ mức ~1GB xuống chỉ còn khoảng ~150MB, đồng thời thu hẹp tối đa bề mặt tấn công của tin tặc.

4.3.2. Quản lý luồng với Docker Compose

Tệp docker-compose.yml được xây dựng để định nghĩa và khởi chạy ứng dụng cùng với các biến môi trường cấu hình cần thiết (như các API Key của Gemini, OpenWeather). Quá trình khởi động toàn bộ nền tảng trên một máy chủ vật lý giờ đây được tự động hóa hoàn toàn chỉ bằng một câu lệnh: `docker-compose up -d --build`.

4.4. Triển khai Hệ thống và Tích hợp Liên tục (CI/CD Production Deployment)

Bên cạnh giải pháp triển khai bằng Docker trên máy chủ riêng (VPS), nhằm tận dụng sức mạnh của Mạng phân phối nội dung toàn cầu (CDN) và cấu trúc Serverless, dự án đã triển khai song song luồng Tích hợp và Phân phối liên tục (CI/CD) thông qua nền tảng **Vercel** - đơn vị chủ quản của framework Next.js. Quá trình triển khai được thiết lập với tên miền chính thức: **dalatvibe.wanghoc.id.vn**.

4.4.1. Quy trình CI/CD và Serverless Deployment

Dự án được kết nối trực tiếp với kho lưu trữ mã nguồn GitHub. Luồng CI/CD hoạt động theo cơ chế tự động:

1. Mỗi khi có mã nguồn mới được Đẩy (Push) lên nhánh chính (main) trên GitHub, Vercel Trigger sẽ tự động bắt tín hiệu và khởi chạy quá trình biên dịch (Build Pipeline) trên các máy chủ đám mây.
2. Trong quá trình build, các script tùy chỉnh được định nghĩa trong `vercel.json` và `package.json` sẽ tự động thực thi lệnh `prisma generate` để chuẩn bị kết nối tới Supabase.
3. Kiến trúc Serverless: Vercel tự động phân rã ứng dụng Next.js. Toàn bộ các trang tĩnh (Static Pages) và hình ảnh được phân tán ra mạng lưới Edge CDN toàn cầu để du khách có thể tải trang với tốc độ cực nhanh. Ngược lại, các thư mục trong `src/app/api/` (bao gồm API gọi Gemini AI, API thời tiết) được tự động đóng gói thành các Serverless Functions (Hàm không máy chủ), chỉ chạy khi có yêu cầu từ người dùng, giúp tối ưu hóa 100% tài nguyên tính toán.

4.4.2. Quản lý Biến môi trường và Chứng chỉ Bảo mật (SSL/TLS)

Để bảo vệ các thông tin nhạy cảm, toàn bộ các khóa kết nối (Supabase `DATABASE_URL`, `JWT_SECRET`, `GEMINI_API_KEY`, `OPENWEATHER_API_KEY`) không được mã hóa cứng (hardcode) mà được khai báo bảo mật trong hệ thống Quản lý Biến môi trường (Environment Variables) của Vercel Project Settings.

Về mặt mạng lưới, tên miền phụ dalatvibe được cấu hình bản ghi CNAME trỏ về hệ thống của Vercel. Ngay khi tên miền được kết nối thành công, Vercel tự động cung cấp, cấu hình và gia hạn Chứng chỉ bảo mật SSL (Let's Encrypt). Quá trình này thiết lập kênh giao tiếp mã hóa HTTPS vững chắc. Mọi dữ liệu trao đổi giữa thiết bị của du khách và hệ thống (bao gồm thông tin đăng nhập, token JWT và nội dung trò chuyện riêng tư với AI) đều được bảo vệ toàn vẹn khỏi các rủi ro đánh cắp dữ liệu hay tấn công trung gian (Man-in-the-middle).

4.5. Ứng dụng Trí tuệ nhân tạo (AI Tools) trong quy trình phát triển

Theo xu hướng hiện đại của ngành kỹ nghệ phần mềm, lập trình viên không chỉ là người gõ mã (coding) mà phải biết khai thác sức mạnh của Trí tuệ nhân tạo để tăng tốc độ phát triển (Rapid Application Development). Trong quá trình thực hiện dự án này, các công cụ AI (như ChatGPT, Gemini CLI, Copilot) đã được sử dụng linh hoạt đóng vai trò như những "trợ lý lập trình cặp" (Pair-programming assistants).

Dưới đây là Bảng phụ lục minh chứng chi tiết (Prompt Log) thống kê các chỉ thị quan trọng nhất đã được sử dụng để hỗ trợ thiết kế kiến trúc, sinh mã nguồn, và gỡ lỗi (debugging) hệ thống:

STT	Phân hệ phát triển	Nội dung câu lệnh đã sử dụng (Prompt)	Kết quả và Đánh giá sự hỗ trợ của AI
1	Thiết kế CSDL & Prisma	"Viết giúp tôi schema.prisma cho dự án du lịch. Cần có các bảng: User, Place, Review, Favorite. Lưu ý bảng Place phải có các trường latitude, longitude, rating, reviewCount và đặc biệt là cờ indoorSuitable kiểu Boolean. Thiết lập quan hệ 1-nhiều và cascade delete chuẩn xác."	Kết quả: AI sinh ra cấu trúc file schema.prisma chuẩn xác với các model và relation đầy đủ. Đánh giá: Tiết kiệm nhiều thời gian thiết kế thủ công, ngăn chặn lỗi cú pháp thiếu sót khóa ngoại khi chạy migration lên Supabase.
2	Bảo mật CSDL (Supabase RLS)	"Tôi đang dùng Supabase (PostgreSQL). Viết các câu lệnh SQL để thiết lập RLS (Row Level Security) cho bảng"	Kết quả: AI cung cấp chính xác 3 câu lệnh CREATE POLICY áp dụng trực tiếp vào bảng quản trị Supabase SQL Editor.

		Review. Yêu cầu: Khách vãng lai được phép đọc (SELECT) tất cả review, nhưng người dùng chỉ được phép xóa (DELETE) hoặc cập nhật (UPDATE) review nếu auth.uid() trùng với trường userId của bản ghi đó."	Đánh giá: Giải quyết thành công bài toán phân quyền phức tạp (NFR-01), đảm bảo an toàn dữ liệu 100% không bị giả mạo sửa bài viết.
3	Tích hợp Logic API (Weather)	"Trong Next.js App Router (TypeScript), hãy viết một Route Handler (GET /api/places/weather-recs) thực hiện: 1. Đọc query param weather. 2. Dùng Prisma truy vấn bảng Place. Nếu weather='rainy' thì lọc indoorSuitable=true và sort theo rating giảm dần. Trả về JSON."	Kết quả: AI trả về đoạn mã route.ts sử dụng NextRequest, có khối try/catch bắt lỗi hợp lệ và cú pháp truy vấn prisma.place.findMany chuẩn xác. Đánh giá: Tối ưu hóa được quy trình lọc dữ liệu theo thuật toán thời tiết đã phân tích ở Chương 3.
4	Trợ lý Ảo (Gemini Stream)	"Tôi cần gọi API Google Gemini bằng thư viện @google/generative-ai trong Next.js API Route. Hãy viết mã để nhận message từ body, kết hợp với một chuỗi System Prompt do tôi định nghĩa, và trả về dữ liệu dưới dạng Streaming (ReadableStream) xuống client."	Kết quả: AI sử dụng GoogleGenerativeAI class, gọi hàm generateContentStream và cung cấp mã bọc (wrapper) trả về một Response HTTP streaming chuẩn. Đánh giá: Rất hữu ích. Việc thiết lập Streaming thủ công phức tạp và dễ lỗi, AI đã cung cấp mã Boilerplate giúp chức năng Chat mượt mà ngay lập tức.

5	Giao diện (Tailwind CSS)	"Viết một component PlaceCard.jsx bằng React và Tailwind CSS. Giao diện cần dạng Thẻ (Card), bo góc, có bóng đổ. Nửa trên là ảnh bao phủ (object-cover). Nửa dưới chứa Tiêu đề, Số sao và nút 'Lưu yêu thích'. Giao diện phải chuẩn Mobile-first."	Kết quả: AI sinh mã JSX sử dụng các lớp utility của Tailwind như flex, flex-col, rounded-xl, overflow-hidden. Đánh giá: Giảm 80% thời gian căn chỉnh CSS thủ công. Thẻ hiển thị đẹp, responsive trên điện thoại ngay từ lần chạy đầu tiên.
6	DevOps (Docker)	"Viết một Dockerfile tối ưu bằng phương pháp Multi-stage build cho dự án Next.js 14 App Router. Cần thu gọn image size bằng cách copy thư mục .next/standalone. Môi trường chạy trên Alpine Linux."	Kết quả: AI cung cấp tệp tin chia rõ 3 giai đoạn deps, builder, runner. Có thiết lập biến môi trường NODE_ENV=production. Đánh giá: Mã chuẩn xác, giúp thu gọn kích thước container xuống còn khoảng 150MB, hệ thống khởi động cực kỳ nhẹ và nhanh.

Bảng 4.1: Minh chứng ứng dụng AI trong phát triển dự án

Việc sử dụng các công cụ Trí tuệ nhân tạo trong quá trình thực hiện đồ án không làm thay thế tư duy và năng lực cốt lõi của lập trình viên, mà đóng vai trò như một bộ tăng tốc (Accelerator) đắc lực. Nhờ có định hướng kiến trúc và thiết kế hệ thống rõ ràng từ trước (ở Chương 3), kỹ sư có thể soạn thảo các câu lệnh (Prompts) mang tính định hướng chuyên môn cao. Qua đó, khai thác được sức mạnh mã hóa của AI để giải quyết nhanh chóng các phần việc lặp đi lặp lại (boilerplate code) hoặc các cấu hình rườm rà (như Dockerfile, PostgreSQL Policies), dành trọn vẹn thời gian và nguồn lực để tập trung nghiên cứu, thiết kế các luồng logic nghiệp vụ phức tạp của nền tảng.

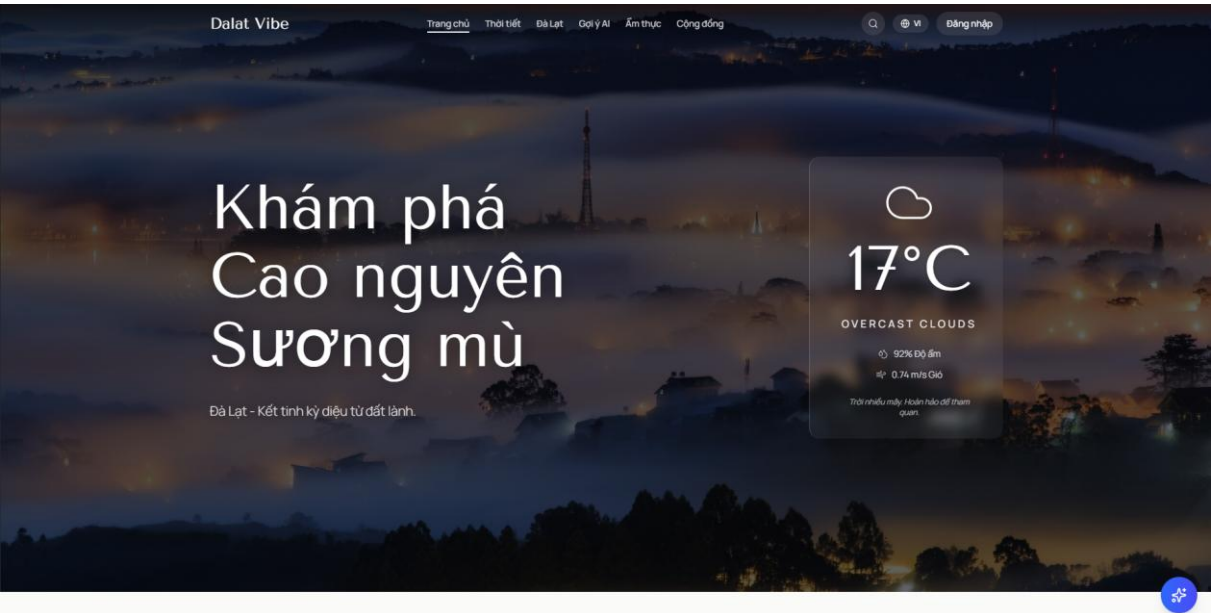
4.6. Kết quả triển khai giao diện người dùng (UI/UX Results)

Sau quá trình lập trình và tích hợp các phân hệ Frontend và Backend, hệ thống Dalat SmartRoute AI đã hoàn thiện giao diện người dùng và đưa vào vận hành thực tế.

Giao diện được thiết kế tuân thủ nguyên tắc Mobile-first (Ưu tiên thiết bị di động) bằng Tailwind CSS, mang lại trải nghiệm mượt mà, tốc độ tải trang nhanh và bố cục trực quan cho du khách. Dưới đây là kết quả triển khai các màn hình chức năng cốt lõi của nền tảng.

4.6.1. Trang chủ và Khối tìm kiếm trung tâm (Hero Section & Search)

Trang chủ là điểm chạm đầu tiên của du khách khi truy cập vào nền tảng. Giao diện được thiết kế nổi bật với khối Hero Banner mang đậm màu sắc đặc trưng của không gian du lịch Đà Lạt. Khối tìm kiếm (Search Bar) được đặt ở vị trí trung tâm, cho phép người dùng dễ dàng nhập từ khóa, kết hợp với các bộ lọc danh mục (Quán Cafe, Lưu trú, Điểm tham quan) để tra cứu thông tin nhanh chóng.

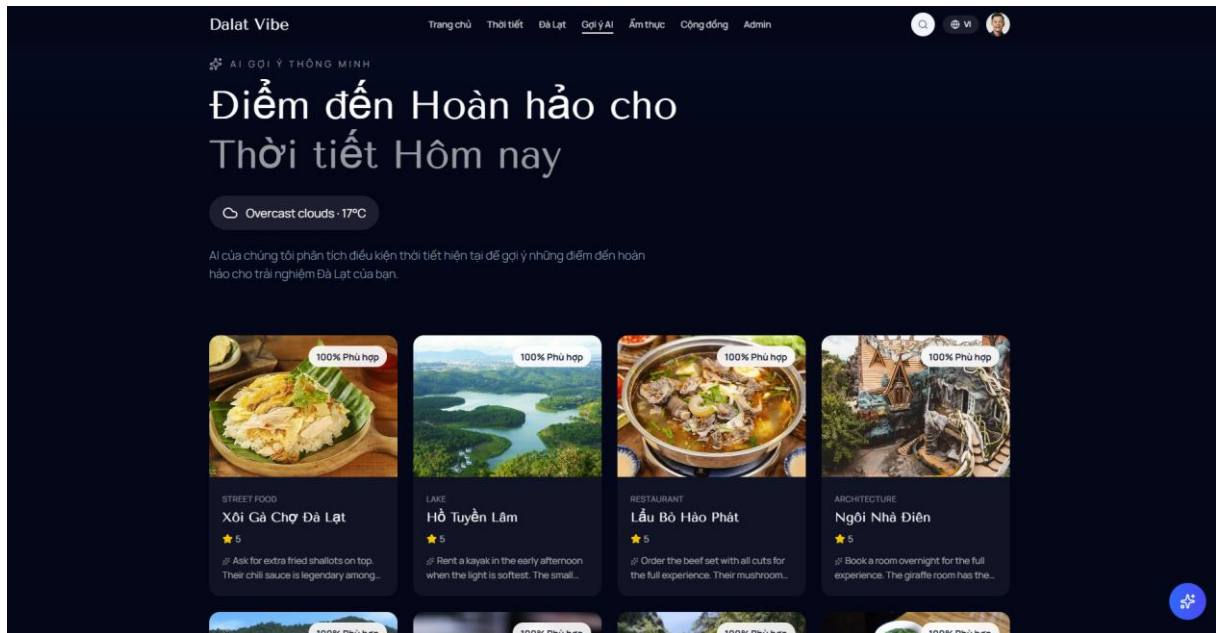


Hình 4.1: Ảnh chụp màn hình Trang chủ (Home Page) của hệ thống

Thanh điều hướng (Navbar) ở phía trên cùng được thiết kế cố định (sticky), tích hợp biểu tượng đăng nhập/đăng ký và trình chuyển đổi ngôn ngữ (Language Switcher), đảm bảo tính khả dụng (Usability) cao trong suốt quá trình người dùng cuộn trang.

4.6.2. Giao diện khám phá và Gợi ý theo thời tiết (Weather-based Recommendations)

Đây là phân hệ hiện thực hóa tính năng nghiệp vụ lõi của đồ án. Giao diện hiển thị các thẻ địa điểm (Place Cards) được bo góc mềm mại, có hiệu ứng đổ bóng (shadow) và ảnh bìa chất lượng cao. Mỗi thẻ hiển thị rõ Tên địa điểm, số sao đánh giá (Rating) và nút lưu Yêu thích (Favorite).

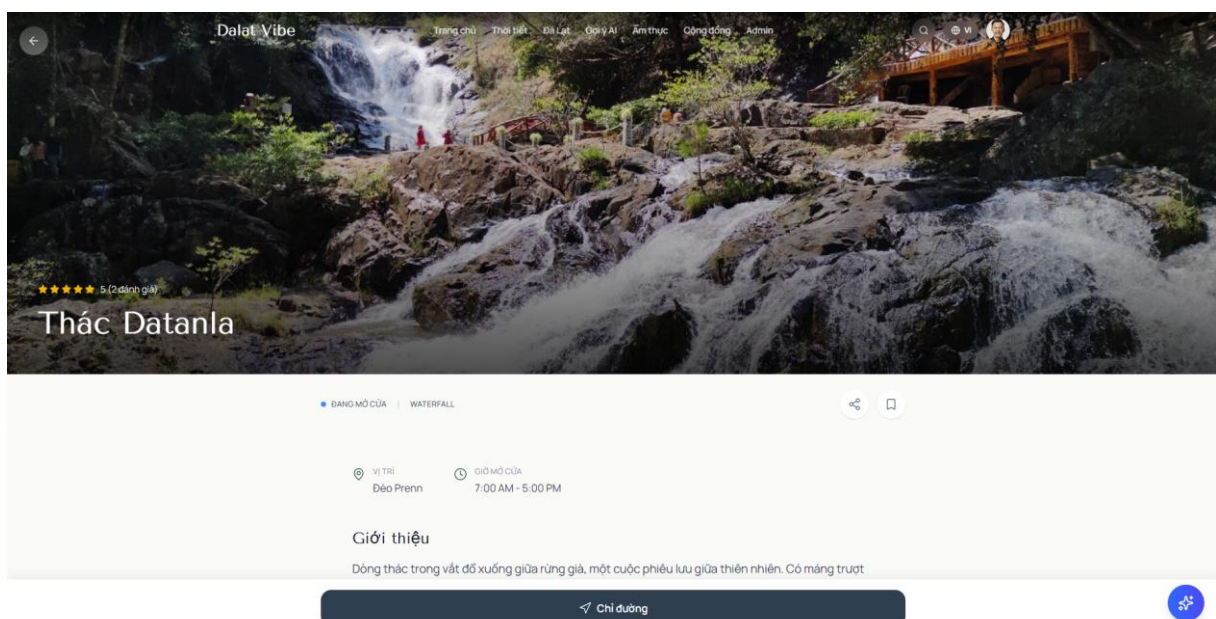


Hình 4.2: Ảnh chụp màn hình trang Danh sách địa điểm (Places List)

Điểm nhân nằm ở khu vực thông báo ngữ cảnh thời tiết. Hệ thống sẽ hiển thị một khối cảnh báo trực quan (Ví dụ: Icon trời mưa kèm nhiệt độ hiện tại lấy từ OpenWeather API). Ngay bên dưới là danh sách các địa điểm đã được bộ lọc indoorSuitable xử lý, ưu tiên hiển thị các không gian trong nhà để đáp ứng tình hình thời tiết thực tế.

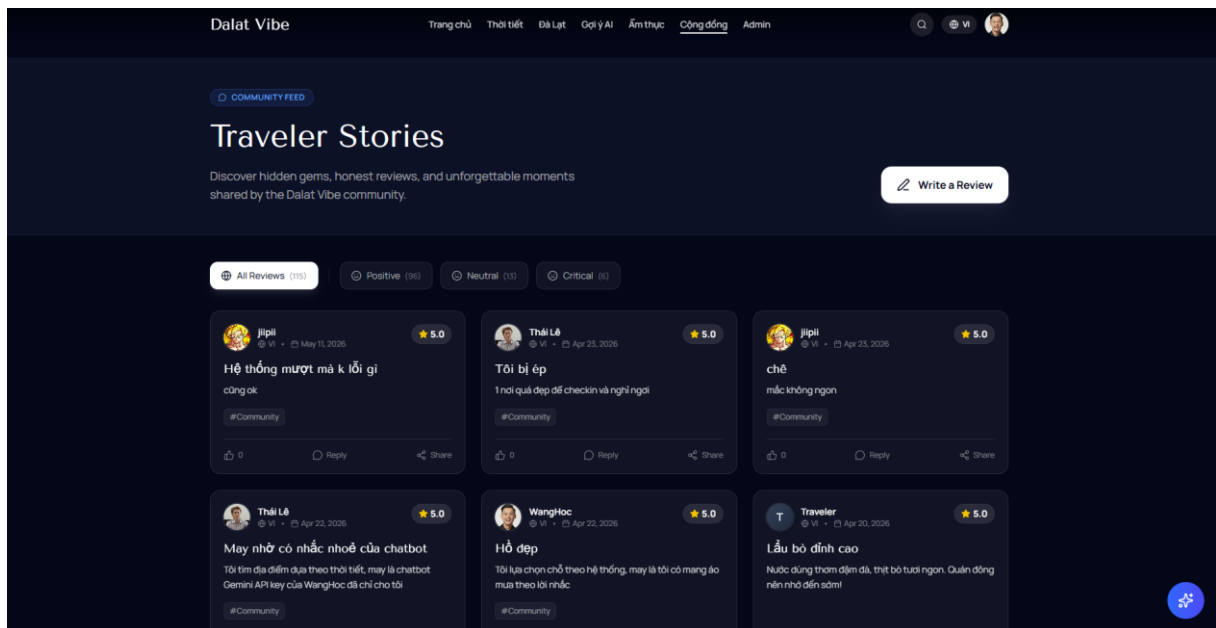
4.6.3. Giao diện Chi tiết địa điểm và Tương tác cộng đồng (Place Details & Reviews)

Khi truy cập vào một địa điểm cụ thể, người dùng được cung cấp một không gian thông tin toàn diện. Giao diện được chia bố cục khoa học: phần hình ảnh tổng quan, phần văn bản mô tả chi tiết, tích hợp bản đồ (Google Maps Link) và trạng thái đóng/mở cửa.



Hình 4.3: Ảnh chụp màn hình trang Chi tiết địa điểm (Detail Page)

Phía dưới thông tin địa điểm là khu vực dành riêng cho Tương tác cộng đồng. Hệ thống hiển thị điểm số trung bình lớn, kèm theo danh sách các bài đánh giá (Reviews) từ người dùng khác.



Hình 4.4: Ảnh chụp màn hình khu vực Bình luận và Đánh giá (Review Form)

Giao diện form viết đánh giá cung cấp công cụ chọn sao (Star Rating) tương tác trực quan. Nút gửi đánh giá chỉ được kích hoạt (enable) khi người dùng đã đăng nhập, qua đó kiểm soát chặt chẽ luồng dữ liệu cộng đồng.

4.6.4. Giao diện Trợ lý ảo thông minh (AI Chatbot Widget)

Tính năng Trợ lý ảo được thiết kế dưới dạng một Widget Chat nổi (Floating Chat Widget) ở góc dưới cùng bên phải màn hình, cho phép du khách có thể mở lên và tương tác ở bất kỳ trang nào mà không làm gián đoạn luồng tra cứu thông tin.

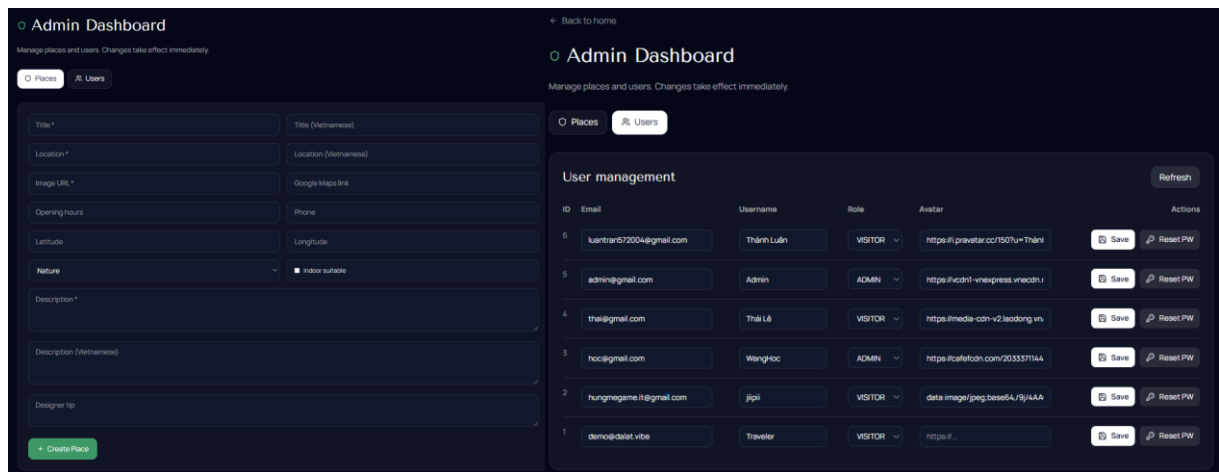


Hình 4.5: Ảnh chụp màn hình của sổ Chatbot AI đang tương tác trả lời câu hỏi của người dùng

Cửa sổ chat hiển thị rõ ràng thông điệp của người dùng (User Message) và luồng phản hồi của AI (AI Response). Nhờ ứng dụng công nghệ Readable Stream, văn bản từ Google Gemini được kết xuất dần dần lên màn hình, tạo hiệu ứng thị giác gõ chữ tự nhiên, mang lại cảm giác thân thiện như đang giao tiếp với một hướng dẫn viên người thật.

4.6.5. Bảng điều khiển Quản trị viên (Admin Dashboard)

Dành riêng cho những tài khoản có vai trò ADMIN, hệ thống cung cấp một trang Bảng điều khiển (Dashboard) ẩn. Giao diện quản trị được thiết kế theo dạng bảng dữ liệu (Data Data), tối ưu hóa cho các thao tác CRUD (Thêm, Xem, Sửa, Xóa).



Hình 4.6: Ảnh chụp màn hình Bảng điều khiển Admin quản lý danh sách địa điểm hoặc người dùng

Tại đây, Quản trị viên có thể dễ dàng quản lý kho dữ liệu địa điểm, thiết lập tọa độ, bật/tắt cờ indoorSuitable cho từng địa điểm, cũng như theo dõi tổng quan số lượng người dùng và các đánh giá trên hệ thống.

CHƯƠNG 5: TỔNG KẾT, ĐÁNH GIÁ VÀ HƯỚNG PHÁT TRIỂN

5.1. Tổng kết những kết quả đạt được

Đồ án đã trải qua một vòng đời phát triển phần mềm (SDLC) hoàn chỉnh, từ khâu khảo sát yêu cầu đến khi triển khai thành công trên môi trường thực tế. Về tổng thể, dự án đã đạt được những thành tựu nhất định trên cả hai phương diện: Nghiệp vụ và Kỹ thuật.

- Về mặt nghiệp vụ: Đề tài đã xây dựng thành công một nền tảng khám phá du lịch mang tính cá nhân hóa cao. Hệ thống giải quyết trực tiếp bài toán "Thời tiết cực đoan ảnh hưởng đến trải nghiệm du lịch" tại Đà Lạt bằng cách tự động hóa quy trình lọc địa điểm theo ngữ cảnh môi trường (trời mưa/trời nắng). Hơn thế nữa, nền tảng đã thiết lập được một không gian tương tác cộng đồng (community-driven) thông qua hệ thống đánh giá (Review), lưu trữ (Favorite) và tư vấn lịch trình linh hoạt bằng Trợ lý ảo AI.
- Về mặt kỹ thuật: Dự án đã làm chủ và ứng dụng thành công một hệ sinh thái công nghệ toàn diện và hiện đại nhất (Modern Tech Stack). Đội ngũ phát triển đã triển khai kiến trúc App Router của Next.js, tích hợp Backend-as-a-Service an toàn với Supabase (PostgreSQL + RLS), thiết kế tầng giao tiếp dữ liệu an toàn với Prisma ORM, và đặc biệt là ảo hóa thành công ứng dụng bằng nền tảng Docker với quy trình CI/CD tích hợp trên Vercel.

5.2. Đánh giá ưu điểm của hệ thống

Quá trình xây dựng và vận hành Dalat SmartRoute AI cho thấy hệ thống sở hữu những ưu điểm vượt trội so với các kiến trúc truyền thống:

1. Kiến trúc phân tầng rõ ràng và độc lập (Separation of Concerns): Hệ thống phân tách rạch ròi giữa Lớp Giao diện (Frontend Components), Lớp Dịch vụ (API Routes) và Lớp Lưu trữ (Prisma & Supabase Database). Sự phân tách này giúp mã nguồn dễ đọc, dễ bảo trì và tạo tiền đề để mở rộng hoặc thay thế bất kỳ module nào (ví dụ: thay đổi giao diện) mà không làm sụp đổ các luồng xử lý dữ liệu lõi.
2. Đột phá với sự tích hợp AI và Dữ liệu ngữ cảnh: Sản phẩm không chỉ là một trang web hiển thị dữ liệu tĩnh (CRUD cơ bản) mà đã trở thành một hệ thống "Sống". Việc kết hợp song song dữ liệu thời gian thực từ OpenWeather API và

năng lực xử lý ngôn ngữ tự nhiên của Google Gemini LLM mang lại trải nghiệm tương tác (UX) ở đẳng cấp cao, vượt xa các nền tảng cảm nang du lịch truyền thống.

3. Mô hình bảo mật đa lớp (Multi-layer Security): Hệ thống được bảo vệ vững chắc từ tầng ứng dụng (JWT-based Authentication) cho đến tầng hạ tầng cơ sở dữ liệu (Role-Based Access Control và Row Level Security của PostgreSQL). Việc ủy quyền quy trình băm mật khẩu (Password Hashing) cho thư viện chuẩn ngành giúp bảo vệ toàn vẹn dữ liệu cá nhân của người dùng.
4. Quy trình DevOps chuẩn mực: Việc áp dụng Docker Multi-stage build giúp giảm thiểu dung lượng container, tăng tốc độ triển khai. Kết hợp với nền tảng Edge Network của Vercel và chứng chỉ SSL/TLS tự động, hệ thống đạt được tính sẵn sàng (Availability) cao.

5.3. Hạn chế và các vấn đề còn tồn đọng

Bên cạnh những kết quả tích cực, do giới hạn về mặt thời gian và nguồn lực của một đồ án môn học, hệ thống vẫn còn một số điểm nghẽn kỹ thuật (Technical Debts) cần được thẳng thắn nhìn nhận và khắc phục:

1. Thiếu vắng cơ chế Bộ nhớ đệm (Caching Mechanism): Hiện tại, các luồng xử lý nặng như truy vấn dữ liệu thời tiết (OpenWeather) và gọi API mô hình AI (Gemini) đều được thực thi trực tiếp trên mỗi yêu cầu của người dùng. Việc không sử dụng bộ đệm (như Redis hoặc Memcached) dẫn đến thời gian phản hồi (Latency) đôi lúc còn cao, đồng thời làm tăng chi phí vận hành (API Cost) nếu lưu lượng truy cập gia tăng đột biến, vì hệ thống phải liên tục gửi yêu cầu ra mạng bên ngoài cho những câu hỏi trùng lặp.
2. Hạn chế về Khả năng quan sát và Giám sát hệ thống (Observability): Dự án chưa được tích hợp các công cụ thu thập log tập trung (Centralized Logging) và theo dõi lỗi thời gian thực (Error Tracking/APM). Hiện tại, nếu một API nội bộ hoặc tiến trình kết nối cơ sở dữ liệu bị sụp đổ trên môi trường Production, quản trị viên không nhận được cảnh báo tự động, gây khó khăn lớn cho việc gỡ lỗi (Debugging).
3. Chưa triển khai Kiểm thử Tự động (Automated Testing): Quy trình đảm bảo chất lượng phần mềm (QA) hiện chỉ phụ thuộc vào phương pháp Kiểm thử thủ công (Manual Testing). Dự án chưa có các bộ Kiểm thử đơn vị (Unit Tests) cho các

hàm xử lý logic hay Kiểm thử tích hợp (Integration Tests) cho các API. Điều này làm tăng rủi ro hồi quy (Regression Bugs) khi bổ sung tính năng mới, khiến mã nguồn trở nên mong manh (fragile) trong dài hạn.

4. Thiếu cơ chế Giới hạn tỷ lệ truy cập (Rate Limiting): Các điểm cuối API (đặc biệt là API tạo tài khoản và API Chat AI) hiện chưa có hàng rào kiểm soát số lượng truy cập (Rate Limiting hoặc Throttling). Đây là một lỗ hổng tiềm ẩn, khiến hệ thống dễ trở thành mục tiêu của các cuộc tấn công Từ chối dịch vụ (DDoS) hoặc bị lạm dụng tài nguyên API, làm cạn kiệt ngân sách dịch vụ của Google Gemini.

5.4. Hướng phát triển trong tương lai

Từ việc phân tích các hạn chế nêu trên và định hướng mở rộng vòng đời sản phẩm (Product Lifecycle), dự án đề xuất các hướng phát triển kỹ thuật và nghiệp vụ trong tương lai như sau:

- Tối ưu hóa Hiệu năng và Chi phí với Redis: Triển khai một cụm In-memory Database (như Redis) để làm lớp Caching trung gian. Kết quả trả về từ OpenWeather (vốn chỉ thay đổi mỗi 1-2 giờ) và các danh sách địa điểm nổi bật sẽ được lưu vào bộ nhớ đệm, giúp giảm tải trực tiếp cho cơ sở dữ liệu Supabase và hạ thời gian phản hồi API xuống mức mili-giây (ms).
- Nâng cấp Hệ thống Giám sát (Monitoring): Tích hợp nền tảng Sentry hoặc Datadog vào ứng dụng Next.js để thu thập toàn bộ Log lỗi, theo dõi hiệu năng Web Vitals và cấu hình hệ thống cảnh báo (Alerts) qua Email/Telegram cho đội ngũ quản trị ngay khi có sự cố.
- Xây dựng Pipeline CI/CD toàn diện: Bổ sung các công cụ như Jest (Unit Test) và Cypress (End-to-End Test) vào chu trình GitHub Actions. Bất kỳ lệnh Pull Request (PR) nào cũng phải vượt qua được toàn bộ các bài kiểm thử tự động và kiểm tra mã (Linting) trước khi được phép triển khai lên Vercel.
- Bảo vệ hệ thống bằng Rate Limiting: Triển khai thuật toán Token Bucket hoặc cấu hình tính năng Rate Limit trên Cloudflare WAF để giới hạn số lượng yêu cầu API từ một địa chỉ IP (Ví dụ: Tối đa 20 câu hỏi AI / 1 giờ / 1 IP), qua đó chống lại hành vi lạm dụng hệ thống.
- Mở rộng Nghiệp vụ và Đa nền tảng: Về mặt sản phẩm, sẽ nghiên cứu xây dựng tính năng "Tự động thiết kế Lịch trình" (Itinerary Builder) – cho phép người dùng

kéo thả các địa điểm do AI gợi ý vào một bản đồ kế hoạch. Đồng thời, đóng gói các API hiện tại để phục vụ cho việc phát triển một ứng dụng di động riêng biệt (Mobile Native App) sử dụng React Native hoặc Flutter trong giai đoạn tiếp theo.

Dự án "Phát triển nền tảng du lịch thông minh dựa theo thời tiết và tích hợp trợ lý ảo" đã hoàn thành trọn vẹn và thể hiện được sự nghiêm túc trong quá trình nghiên cứu, ứng dụng các công nghệ lập trình web hiện đại nhất. Bất chấp những giới hạn về mặt thời gian, đề án đã chứng minh được tính khả thi cao của việc ứng dụng Trí tuệ nhân tạo Sinh tạo (Generative AI) vào thực tiễn ngành dịch vụ, mở ra một hướng tiếp cận mới trong việc xây dựng các nền tảng thông minh lấy người dùng và ngữ cảnh làm trung tâm. Những hạn chế và kinh nghiệm rút ra từ dự án này sẽ là cơ sở quý báu để tiếp tục nghiên cứu và hoàn thiện các sản phẩm phần mềm quy mô lớn hơn trong tương lai.

TÀI LIỆU THAM KHẢO

- [1] Vercel, "Next.js: The React Framework for the Web," *Next.js Documentation*, 2026. [Online]. Available: <https://nextjs.org/docs>
- [2] Supabase, "The Open Source Firebase Alternative," *Supabase Documentation*, 2026. [Online]. Available: <https://supabase.com/docs>
- [3] Prisma Data, "Prisma ORM Documentation," *Prisma Developer Docs*, 2026. [Online]. Available: <https://www.prisma.io/docs/orm>
- [4] Google, "Gemini API Documentation," *Google AI for Developers*, 2026. [Online]. Available: <https://ai.google.dev/docs>
- [5] OpenWeather, "Current weather data API," *OpenWeatherMap API Portal*, 2026. [Online]. Available: <https://openweathermap.org/current>
- [6] Microsoft, "TypeScript Documentation," *TypeScript Handbook*, 2026. [Online]. Available: <https://www.typescriptlang.org/docs>
- [7] Tailwind Labs, "Tailwind CSS Documentation," *Tailwind CSS*, 2026. [Online]. Available: <https://tailwindcss.com/docs>
- [8] PostgreSQL Global Development Group, "Row Security Policies," *PostgreSQL 16 Documentation*, 2026. [Online]. Available: <https://www.postgresql.org/docs/current/ddl-rowsecurity.html>
- [9] Docker Inc., "Docker Documentation," *Docker Docs*, 2026. [Online]. Available: <https://docs.docker.com>